

APUNTESWEB · INGENIERÍA INFORMÁTICA

COMDIS

Tercer curso · 1.º cuatrimestre · 11 temas

JULIO DE 2026 · [HTTPS://ADRIANQL5.GITHUB.IO/APUNTESWEB/](https://adrianql5.github.io/apuntesweb/)

1. Paradigmas de la Computación Distribuida

1.1 ¿Qué es esto y por qué nos importa?

1.1.1 El Concepto de Paradigma

Antes de hablar de cables y servidores, definamos el término. Un paradigma es simplemente un **patrón o modelo**. En computación, dada la complejidad enorme de conectar máquinas, usamos estos modelos para no tener que reinventar la rueda cada vez que escribimos software.

1.1.2 Aplicación Distribuida vs Convencional

Imagina un programa en tu portátil (Word, Calculadora). Eso es convencional. Una **aplicación distribuida** tiene dos características que la hacen especial y difícil:

1. **Comunicación:** Necesita que dos o más procesos independientes (en máquinas distintas o la misma) intercambien datos.
2. **Sincronización:** No basta con enviar datos; hay que coordinar *cuándo* se envían y reciben.

Concepto Clave: Abstracción La abstracción es el arte de **esconder los detalles sucios**.

- *Ejemplo:* Cuando conduces, usas el volante (abstracción alta). No inyectas gasolina manualmente en los cilindros (abstracción baja). En sistemas distribuidos, buscamos herramientas que oculten la complejidad de la red al programador.

1.2 Los Cimientos (Nivel Bajo)

Aquí es donde todo empieza. Es el nivel de abstracción más bajo.

El Paso de Mensajes (Message Passing)

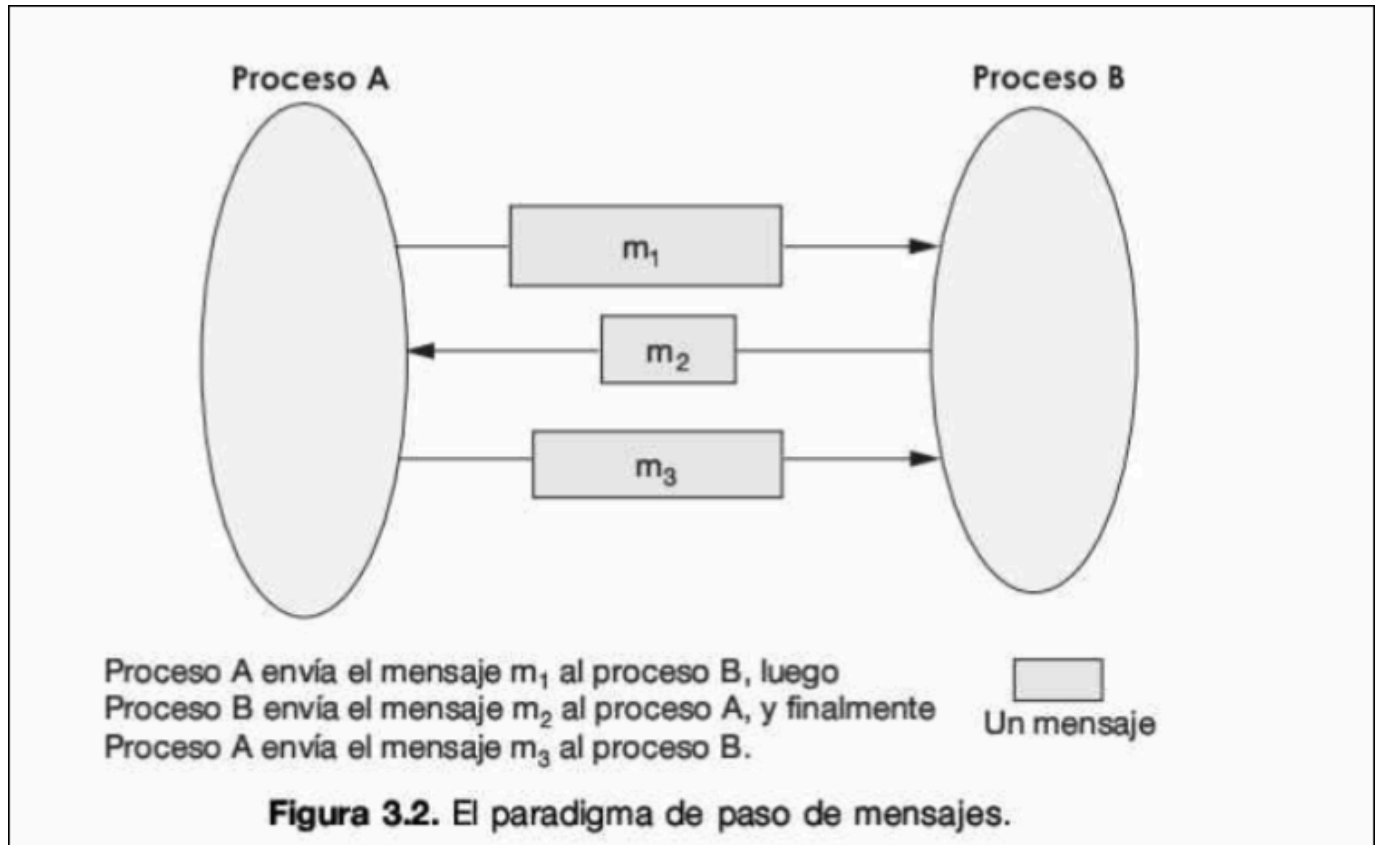
Concepto Principal: es el paradigma fundamental. Cualquier cosa que ocurra en Internet, en el fondo, es un mensaje llendo de A a B. Es como pasarse notitas en clase. Tú escribes un papel (petición), se lo das a un compañero y esperas que te devuelva otro papel (respuesta).

Detalle Técnico:

- Se basa en operaciones de Entrada/Salida (I/O) similares a escribir en un archivo.
- **Primitivas básicas:** `send` (enviar) y `receive` (recibir)
- Si la conexión es continua (como una llamada telefónica), usamos `connect` y `disconnect`

- **La herramienta real:** esto se implementa habitualmente con **Sockets** a nivel de Sistema operativo.

Cuando tu navegador pide una web, por debajo está abriendo un socket y pasando mensajes. Un "problema" a tener en cuenta es que los procesos están fuertemente acoplados (ambos deben de estar activos al mismo tiempo para que se produzca la comunicación).



INFO

Una **primitiva** es la operación más básica y elemental que te ofrece un sistema o lenguaje, es un ladrillo indivisible con el que construyes cosas más grandes.

Un **socket** es un **enchufe** de software. Es el punto final (endpoint) de una comunicación bidireccional entre dos programas a través de la red. Imagina que tu ordenador es un bloque de pisos. El **Socket** es la puerta de un apartamento específico (identificado por una IP y un Puerto). Si quieres enviar una carta a otro edificio, necesitas saber la dirección del edificio (IP) y el número de puerta (Puerto) del destinatario.

Usamos `connect` / `disconnect` cuando necesito fiabilidad absoluta y un canal estable. Antes de enviar datos, debes descolgar el teléfono (`connect`) y al terminar colgar (`disconnect`). Garantiza que los datos llegan y en orden (**Servicio orientado a conexión**). No se usa (**servicio no orientado a conexión**) cuando la velocidad es más importante que la fiabilidad, o cuando envías mensajes sueltos. Solo se hace `send` y `receive` sin llamar antes. `Connect` tarda tiempo.

1.3 Arquitecturas Básicas

Una vez sabemos pasar mensajes, ¿cómo organizamos a los participantes?

1.3.1 Cliente-Servidor (Client-Server)

Concepto Principal: es el modelo más conocido y **asimétrico**. Hay una clara división de jerarquía. Piensa en un restaurante:

- **Servidor (Camarero):** Espera pasivamente a que alguien le llame
- **Cliente (Comensal):** Inicia la conversación pidiendo algo (request) y espera la comida (response)

La arquitectura cliente-servidor es **asimétrica** puesto que cada nodo que interviene en la arquitectura tiene funciones, objetivos y responsabilidades diferentes. Hay una clase diferencia jerárquica y funcional:

- **Cliente (rol activo):** inicia la comunicación y realiza peticiones al servidor
- **Servidor (rol pasivo):** espera pasivamente a recibir peticiones de los clientes para poder procesarlas.

El objetivo que se persigue es proporcionar una arquitectura de red en la que se puedan ofrecer servicios facilitando la sincronización de eventos y permite que los clientes sean **más** ligeros al centralizar los recursos en el servidor.

1.3.2 Peer-to-Peer (P2P)

Concepto Principal: Arquitectura **simétrica** y descentralizada. Todos los nodos son iguales ("peers"). Es una mesa redonda de trabajo o una comuna. No hay jefes. Si tú tienes un bolígrafo y yo lo necesito, me lo das. Si yo tengo una goma y tú la necesitas, te la doy. A veces actúo como servidor (te doy cosas) y a veces como cliente (te pido cosas).

Detalle Técnico:

- Ideal para compartir recursos (ficheros, ciclos de CPU) de forma escalable
- **Híbridos:** a veces se usa un servidor central solo como directorio (para saber quién tiene qué), como hacía el Napster original, pero la transferencia es directa entre usuarios.

Se usa en BitTorrent, Videoconferencias, Blockchain, aplicaciones de chat, etc.

INFO

Descentralizado significa que **no existe un punto único de fallo ni una autoridad suprema**. En un sistema centralizado, si tiras el servidor central, nadie puede operar.

En P2P la red es una malla, si desconectas a 100 usuarios, los otros 1000 siguen comunicándose entre ellos.

1.4 Subiendo de Nivel: Ocultando la Red (Middleware)

Aquí intentamos que el programador se olvide de que existen cables de red. Queremos programar como si todo estuviera en mi ordenador".

INFO

El middleware es una capa de software que actúa como intermediario entre 2 procesos independientes (aplicaciones, hardware, SO ...). Es un mecanismo útil para el desacoplamiento y la abstracción a la hora de desarrollar aplicaciones distribuidas.

1.4.1 RPC (Remote Procedure Call)

Concepto Principal: Abstracción que permite llamar a una función que está en otro ordenador como si fuera una función local. Imagina que en tu código escribes `calcularSuma(5, 10)`. Tú no sabes si esa suma la hace tu CPU o superordenador en Japón. RPC se encarga de enviar el 5 y el 10 a Japón esperar, y devolverte 15.

Detalle Técnico:

- Usa unos componentes llamados **Stubs**. El stub es una pieza de código falsa en tu ordenador que finge ser la función remota, su único trabajo es empaquetar los datos y enviarlos (ya lo veremos más adelante)

1.4.2 RMI (Remote Method Invocation) y Objetos Distribuidos

Concepto Principal: Es la evolución de RPC para la **Programación Orientada a Objetos**

Diferencia con RPC: En RPC llamas a funciones (acciones). En RMI, invocas a métodos de un **Objeto** que vive en otra máquina. Puedes pasar objetos enteros como argumentos.

1.4.3 ORB (Object Request Broker)

Concepto Principal: El traductor universal o intermediario de objetos

¿Qué pasa si quiero que un objeto escrito en C++ en Windows hable con un objeto escrito en Java en Linux? RMI (de Java) no basta. La solución es **ORB**, que es un middleware (intermediario) que redirige peticiones entre objetos heterogéneas (distintos lenguajes/plataformas). El estándar famoso es el **CORBA**.

1.5 Desacoplamiento: Sistemas de Mensajes (MOM)

A veces no queremos respuesta inmediata. A veces queremos "dejar un recado".

Concepto Principal: Message-Oriented Middleware (MOM). Un intermediario gestiona colas de mensajes, permitiendo comunicación **asíncrona**.

INFO

El desacoplamiento consiste en eliminar las dependencias rígidas entre dos sistemas.

Podemos diferenciar 2 tipos:

- **Temporal:** el emisor y el receptor **no necesitan estar conectados a la vez**. Yo envío el mensaje a las 10:00, me desconecto, y tú lo recibes a las 14:00.
- **Espacial:** el emisor no necesita saber la dirección IP o identidad exacta del receptor, solo envía el mensaje a un buzón.

1.5.1 Modelo Punto a Punto

Funcionamiento: Un emisor deja un mensaje en una cola. El receptor lo recoge cuando puede. Pensadlo como el correo electrónico.

Ventaja: Desacoplamiento total. Si el **receptor** está apagado, el mensaje espera en la cola. No se pierde.

1.5.2 Modelo Publica/Suscribe (Publish/Subscribe)

Funcionamiento: Basado en **Eventos**.

- Un proceso (Publisher) dice: "¡Ha marcado Rodrygo Ganges!" (Evento)
- El sistema distribuye este mensaje a todos los que se hayan "suscrito" al tema "Fútbol"

Por ejemplo, notificaciones de Youtube, Feeds de Twitter, Grupos de Whatsapp.

1.6 Paradigmas Avanzados (Conceptos Futuristas)

Estos modelos son más específicos, pero interesantes.

1.6.1 Servicios de Red

Los servicios se "registran" en un directorio. El cliente busca en el directorio "necesito imprimir", y el directorio le da la dirección de la impresora disponible. Es autoconfigurable.

1.6.2 Espacio de Objetos

Imagina una **pizarra compartida** en la nube. Los productores escriben datos en la pizarra. Los consumidores leen o toman datos de la pizarra. Es la abstracción más alta: no sabes quien consume tu dato, solo lo dejas ahí. Ejemplo: JavaSpaces

1.6.3 Agentes Móviles

En vez de mover los datos hacia el programa, **movemos el programa hacia los datos**. Un "agente" viaja de servidor en servidor realizando tareas.

Imagina que quieres buscar una foto específica en una base de datos de la NASA de 50 Petabytes.

- *Método tradicional:* Te descargas los 50 Petabytes (tardas años) y tu programa busca la foto.
- *Método Agente:* Envías tu pequeño programa (Agente) de 5KB al servidor de la NASA. El programa se ejecuta allí (localmente), busca la foto rapidísimo (porque está al lado del disco duro) y solo te envía de vuelta la foto encontrada. Ahorras un ancho de banda inmenso.

El Agente: Es un programa autónomo. Decide cuándo irse al siguiente nodo del itinerario. Lleva consigo su "equipaje" (el estado actual de su ejecución).

1.6.4 Paradigma de aplicaciones colaborativas

Este paradigma modela cómo trabajamos los humanos en equipo. Los procesos no solo **hablan**, si no que cooperan para manipular un estado compartido en tiempo real.

Componentes clave:

- Multicasting: Enviar un dato a *muchos* a la vez (ej: cuando escribes una letra en Google Docs, se envía "a la vez" a todos los que miran el documento).
- Pizarras Virtuales (Whiteboards): Es la memoria compartida visual. Todos pueden leer y escribir

Si dos personas escriben en la misma línea a la vez, ¿quién gana? Las aplicaciones colaborativas necesitan algoritmos complejos (como Relojes Lógicos, que veremos más adelante) para ordenar estos eventos y que todos vean el mismo documento final.

Ejemplo Real: Google Docs, Trello, Slack, Pizarras de Zoom.

1.7 Resumen y Comparativa

Nivel de Abstracción	Paradigma	¿Qué intercambian?	Ejemplo Real
Bajo (Cimientos)	Paso de Mensajes	Bytes/Datos crudos	Sockets, TCP/IP
Medio	RPC / RMI	Llamadas a Funciones/Métodos	Java RMI, gRPC
Medio	Mensajería (MOM)	Mensajes Asíncronos	RabbitMQ, Kafka
Alto	Objetos Distribuidos / ORB	Objetos complejos	CORBA
Muy Alto	Espacio de Objetos	Datos en pizarra compartida	JavaSpaces

2. El API de Sockets

2.1 Introducción

Antes de profundizar en el código y los protocolos, es vital situar este tema en el mapa de la asignatura. En la jerarquía de abstracción de la comunicación distribuida, nos encontramos en el **Nivel 0: Paso de Mensajes**.

- **Baja Abstracción (Sockets):** Trabajamos directamente con el transporte de bytes entre procesos. Tú defines *cómo* se envían los datos, *cuándo* se leen y *qué* estructura tienen. Es la base de la pirámide.
- **Arquitecturas Básicas:** Utilizando sockets, construimos los paradigmas **Cliente-Servidor** (centralizado) y **P2P** (descentralizado).
- **Alta Abstracción (Futuros temas):** Mecanismos como RPC (Remote Procedure Call), RMI (Remote Method Invocation) o los ORB (Object Request Brokers) son "capas de azúcar sintáctico" y gestión automática que se construyen **sobre** el API de Sockets. Entender sockets es entender la "fontanería" que hace posible todo lo demás

2.2 Conceptos Fundamentales

El **API de Sockets** es la interfaz de programación estándar para la comunicación entre procesos (IPC - *Inter-Process Communication*).

INFO

API significa Application Programming Interface. Imagina que el SO es una fortaleza cerrada que controla el hardware (la tarjeta de red) . Tú, como programador no puedes tocar los cables ni los voltajes directamente. El API de sockets es el conjunto de funciones estandarizadas que el SO te ofrece para pedirle servicios de red.

Un **Socket** se define conceptualmente como un **punto final (endpoint) de comunicación**. Es una abstracción que permite a un proceso enviar y recibir mensajes hacia/desde otro proceso, ya sea en la misma máquina o a través de una red.

Los sockets operan sobre la pila de protocolos **TCP/IP**, que es independiente del fabricante y soporta desde redes locales (LAN) hasta redes globales (WAN) como internet.

Para que dos procesos se comuniquen, necesitamos identificar de forma única quién habla con quién. Esto se logra mediante una **asociación**, que es una tupla de 5 elementos:

- **Protocolo:** TCP o UDP.
- **Dirección IP Local (32 bits):** Identifica la máquina origen.
- **Puerto Local (16 bits):** Identifica el proceso origen dentro de la máquina.
- **Dirección IP Remota (32 bits):** Identifica la máquina destino.
- **Puerto Remoto (16 bits):** Identifica el proceso destino.

2.3 Protocolos de Transporte: UDP vs TCP

El API de Sockets nos permite elegir cómo queremos transportar nuestros datos. Esta elección define la fiabilidad y el rendimiento de nuestra aplicación distribuida.

2.3.1 UDP (User Datagram Protocol)

Es un protocolo **no orientado a conexión**. Se basa en el envío de **mensajes** discretos (datagramas).

Características:

- **No fiable:** no garantiza que los datos lleguen, ni que lleguen en orden
- **Límites de mensaje:** se respetan las fronteras. Si envías 1 paquete de 500 bytes, el receptor recibe 1 paquete de 500 bytes
- **Sin QoS (Quality of Service):** no garantiza ancho de banda ni latencia
- **Simplicidad:** al tener menos sobrecarga de control, es muy rápido

Uso ideal: aplicaciones donde la velocidad prima sobre la exactitud, como videoconferencias o telefonía IP (si se pierde un frame de video, no importa, pero no podemos esperar a retransmitirlo).

2.3.2 TCP (Transmission Control Protocol)

Es un protocolo **orientado a conexión**. Antes de enviar datos, se establece un canal virtual entre emisor y receptor.

Características:

- **Fiabilidad Total:** Garantiza la entrega correcta y el orden secuencial de los paquetes. Si algo se pierde, se retransmite automáticamente.
- **Stream (Flujo de Bytes):** No hay fronteras de mensaje. Si envías 3 paquetes de 10 bytes, el receptor podría leer 30 bytes de golpe o 15 y 15. Es como una tubería de agua.
- **Bidireccional:** Ambos lados pueden leer y escribir simultáneamente.

Uso ideal: Aplicaciones que requieren integridad de datos total: Web (HTTP), Correo (SMTP), Transferencia de archivos (FTP), Telnet .

2.4 Implementación en Java: Sockets Orientados a Conexión (TCP)

En el modelo **Stream** (flujo) , la comunicación es exclusiva entre dos procesos conectados. Un socket stream no sirve para hacer "broadcast" a muchos procesos a la vez.

INFO

Stream (Tubo/TCP): Es un **flujo continuo de bytes** sin fronteras internas.

- Si tú envías "HOLA" y luego "MUNDO" por un stream, el receptor ve simplemente un chorro de bytes: `H O L A M U N D O`.
- El receptor no sabe si lo enviaste todo junto o en dos partes.
- **Garantía:** Es una conexión fiable, ordenada y bidireccional, similar a las tuberías (pipes) de Unix.

2.4.1 Arquitectura Cliente-Servidor en TCP

El servidor debe estar activo antes que el cliente. El proceso se basa en dos tipos de sockets en el lado del servidor:

- **Socket de Conexión (`ServerSocket`):** solo se encarga de esperar clientes en un puerto (hacer `listen()`)
- **Socket de Datos (`Socket`):** cuando llega un cliente, el `ServerSocket` lo acepta y **crea un nuevo socket** dedicado exclusivamente a hablar con ese cliente. Esto permite al servidor seguir aceptando nuevos clientes en el socket de conexión original.

2.4.2 Diagrama de Flujo y Primitivas

- **Servidor:** Crea el socket (`socket()`), lo ata a un puerto (`bind()`), y se pone a escuchar (`listen()`)
- **Servidor:** llama a `accept()` . Esta es una **operación bloqueante**. El proceso servidor se duerme hasta que alguien llame a la puerta.
- **Cliente:** crea su socket y solicita conexión (`connect()`) a la IP/Puerto del servidor
- **Handshake:** se establece la conexión. El método `accept()` del servidor despierta y devuelve un nuevo objeto socket para la comunicación.
- **Intercambio:** ambos usan flujos de entrada/salida (`InputStream/OutputStream`) para `read()` y `write()`
- **Cierre:** se usa `close()` para terminar la conexión

INFO

Todas estas operaciones que aparecen entre paréntesis son proporcionadas por Unix, si haces en un terminal `man funcion`, te pone la info. Son funciones de la API de sockets.

2.4.3 Código Java Esencial (TCP)

En Java, usamos `java.net.ServerSocket` y `java.net.Socket`.

Servidor:

```
ServerSocket serverSocket = new ServerSocket(puerto); // Puerto de escucha
Socket incoming = serverSocket.accept(); // BLOQUEANTE: Espera cliente
// Una vez aquí, tenemos conexión. Obtenemos los streams:
DataInputStream in = new DataInputStream(incoming.getInputStream());
DataOutputStream out = new DataOutputStream(incoming.getOutputStream());
// Leer y Escribir
out.writeByte(5);
System.out.println(in.readByte());
```

INFO

Java encapsula la complejidad de las llamadas al sistema originales de Unix. Cuando haces `ServerSocket servidor = new ServerSocket(8080);`. Por debajo ocurren 3 cosas:

- `socket()`: pide al SO que cree un descriptor de archivo para un nuevo socket
- `bind(8080)`: se vincula este socket a la dirección IP local y al puerto 8080.
- `listen()`: se marca el socket como pasiva, indicando que esté preparado para encolar conexiones entrantes

Cliente:

```
Socket socket = new Socket("direccion_servidor", puerto asignado en servidor); //
Intenta conectar
// Si no hay excepción, estamos conectados.
```

2.4.4 Código Java Multihilo (TCP)

El problema es que mientras el servidor habla con el "Cliente A", el "Cliente B" tiene que esperar en la cola. Si el Cliente A tarda 1 hora, el Cliente B se aburre y se va.

Para solucionar esto, usamos **Hilos (Threads)**. El objetivo es que el hilo principal (`main`) solo se dedique a recibir gente en la puerta, y delegue la conversación a hilos trabajadores. De todas formas los hilos los explicaré al final de este pdf.

Estrategia: "El Conserje y los Botones"

1. **Hilo Principal (El Conserje):** Su único trabajo es estar en el bucle `accept()`.
2. **Hilos Trabajadores (Los Botones):** Cuando llega un cliente, el conserje crea un nuevo hilo, le pasa el socket del cliente y vuelve a la puerta a esperar al siguiente.

NOTA

Los hilos se explican con más detalle al final del tema

Servidor:

```
import java.net.*;
import java.io.*;

public class ServidorMultihilo {
    public static void main(String[] args) {
        try {
            // 1. Constructor: Hace socket() + bind() + listen()
            ServerSocket serverSocket = new ServerSocket(8189);
            System.out.println("Servidor iniciado. Esperando clientes...");

            while (true) {
                // 2. accept(): BLOQUEANTE. Espera a un cliente.
                // Cuando llega alguien, retorna un socket DEDICADO a ese cliente.
                Socket socketCliente = serverSocket.accept();

                // 3. MULTITHILO: En lugar de atenderlo aquí, creamos un hilo.
                System.out.println("Cliente conectado. Creando hilo para
atenderlo.");

                // Creamos una instancia de nuestro "Manejador" (ver abajo)
                ManejadorCliente tarea = new ManejadorCliente(socketCliente);
                Thread hilo = new Thread(tarea);

                // 4. Arrancar el hilo.
                // Esto libera al 'main' para volver arriba al 'accept()'
                inmediatamente.
                hilo.start();
            }
        }
    }
}
```

```

    }
    } catch (IOException e) {
        e.printStackTrace();
    }
}
}

```

Cliente:

```

// Implementamos Runnable para que pueda ser ejecutado por un Thread
class ManejadorCliente implements Runnable {
    private Socket miSocket; // Referencia al socket exclusivo de ESTE cliente

    public ManejadorCliente(Socket s) {
        this.miSocket = s;
    }

    @Override
    public void run() {
        try {
            // Aquí va toda la lógica de read/write que antes tenías en el main
            // [cite: 510-511]
            DataInputStream in = new DataInputStream(miSocket.getInputStream());
            DataOutputStream out = new DataOutputStream(miSocket.getOutputStream());

            // Ejemplo: Eco
            String mensaje = in.readUTF(); // Bloquea solo a ESTE hilo
            System.out.println("Recibido: " + mensaje);

            // Simular proceso lento (ej. consulta a base de datos)
            Thread.sleep(2000);

            out.writeUTF("Eco: " + mensaje);

        } catch (Exception e) {
            e.printStackTrace();
        } finally {
            try {
                // IMPORTANTE: Cerrar el socket del cliente al terminar
                miSocket.close();
            } catch (IOException e) { e.printStackTrace(); }
        }
    }
}

```

```
}  
  
}
```

2.5 Implementación en Java: Sockets No Orientados a Conexión (UDP)

En el modelo **Datagrama**, no hay conexión persistente. Cada paquete es independiente y lleva consigo la dirección del destino. Son como cartas, envías una, luego otra.

2.5.1 Clases Clave

- `DatagramSocket`: es el buzón, sirve tanto para enviar como para recibir. Se liga a un puerto local (`bind()`)
- `DatagramPacket`: es la carta. Contiene los datos (array de bytes) y, si es para enviar, la dirección IP y puerto del destinatario.

2.5.2 Flujo de Comunicación

Al contrario que en TCP, aquí no hay `accept` ni `connect` (en el sentido estricto de establecimiento de sesión).

- **Receptor(Servidor)**: crea socket en puerto conocido. Llama a `receive()`. Se bloquea hasta que llega un paquete.
- **Emisor(Cliente)**: crea paquete con datos+dirección destino. Llama a `send()`
- **Receptor**: `receive()` rellena el paquete con los datos y la dirección de quien lo envió (util para responder)

INFO

`receive()` usa por debajo la función de Unix `recvfrom`

Peligro de Concurrencia: En UDP, si múltiples procesos envían datos al mismo socket receptor simultáneamente, los mensajes se intercalan en la cola de recepción sin orden predecible. El receptor debe gestionar esta "mezcla".

2.5.3 Código Java Esencial (UDP)

Receptor:

```
DatagramSocket ds = new DatagramSocket(2345); // Escuchar en puerto 2345  
byte[] buffer = new byte[1024];
```

```
DatagramPacket dp = new DatagramPacket(buffer, buffer.length);
ds.receive(dp); // BLOQUEANTE
// Datos recibidos en buffer. Info del remitente en dp.getAddress()
```

INFO

`DatagramSocket ds = new DatagramSocket(2345);` por debajo llama a `socket()` y a `bind()`, no usa ni `listen()` ni `accept()`

Diagrama Mental de la Diferencia

- **Constructor TCP (`ServerSocket`):** "Contrato una línea telefónica (`socket`), me asignan el número (`bind`) y me siento a esperar que suene para descolgar (`listen` + `accept`)."
- **Constructor UDP (`DatagramSocket`):** "Instalo un buzón en la calle (`socket`) y le pinto mi número de casa (`bind`). Ya está. El cartero puede tirar cartas dentro cuando quiera."

Emisor:

```
DatagramSocket ds = new DatagramSocket(); // Puerto aleatorio disponible
InetAddress receiverHost = InetAddress.getByName("localhost");
byte[] data = "Hola".getBytes();
DatagramPacket dp = new DatagramPacket(data, data.length, receiverHost, 2345);
ds.send(dp); // Envío "fuego y olvido"
```

Constructores de `DatagramPacket`:

```
DatagramPacket(byte[] data, int length, InetAddress receiver, int port);

DatagramPacket(byte[] data, int length);
```

El primero sirve para enviar una paquete UDP, el parámetro data almacena el contenido del mensaje, `length` indica el tamaño del mensaje y la dirección y el puerto se corresponden con los datos del receptor del mensaje.

El segundo sirve para recibir paquetes UDP, data es la variable que almacena el mensaje recibido y length indica el tamaño máximo del mensaje a recibir.

2.6 Técnicas Avanzadas y Robustez

2.6.1 Gestión de Bloqueos (Timeouts)

Por defecto, leer de un socket (`read` o `receive`) congela el programa eternamente si no llegan datos. Para evitar "cuelgues" indefinidos en sistemas distribuidos reales, debemos programar un **Timeout**.

- Método: `socket.setSoTimeout(int milisegundos)`.
- Si expira el tiempo, se lanza una excepción `InterruptedException`, permitiendo al programa recuperar el control.

2.6.2 Multidifusión (Multicast)

La multidifusión permite enviar un solo paquete y que sea recibido por un grupo de ordenadores, optimizando el ancho de banda. Se basa en **UDP**.

Funcionamiento:

- El emisor envía a una IP de clase D.
- Los receptores deben unirse explícitamente al grupo usando `joinGroup()` sobre un `MulticastSocket`.
- Los routers con capacidad multicast replican los paquetes solo hacia las redes donde hay miembros del grupo.
- El **TTL (Time To Live)** controla cuántos saltos de router puede atravesar el paquete, limitando su alcance (local vs internet)

INFO

Una IP de Clase D es una dirección lógica de 32 bits que comienza por `1110`, utilizada para identificar un **grupo de interés** compartido en lugar de una máquina física específica. Direcciones que empiezan en un rango entre [224-239].

NOTA

Broadcast: difusión a todos los dispositivos de una red **Multicast:** difusión a todos los dispositivos de una subred

2.7 Hilos en Java

Un **hilo** es una secuencia de control dentro de un proceso que ejecuta instrucciones de manera independiente.

- Los procesos son *entidades pesadas*: requieren llamadas al sistema y cambios de contexto costosos.
- Los hilos son *entidades ligeras*: comparten el espacio de memoria del proceso y su cambio de contexto es menos costoso.

2.7.1 Creación de Hilos

En Java, existen dos formas principales de definir un hilo. En ambos casos, el código que ejecuta el hilo se define dentro del método `run()`, pero el hilo se lanza llamando a `start()`.

Heredando de la clase `Thread`

Se crea una clase hija de `Thread` y se sobrescribe el método `run()`

```
public class MiHilo extends Thread {
    public void run() {
        // Código del hilo
    }
}
// Uso:
Thread hilo = new MiHilo();
hilo.start();
```

Implementando la interfaz `Runnable`

Útil si tu clase ya hereda de otra. Se implementa `Runnable` y se pasa la instancia al constructor de `Thread`.

```
public class MiTarea implements Runnable {
    public void run() {
        // Código
    }
}
// Uso:
MiTarea tarea = new MiTarea();
Thread hilo = new Thread(tarea);
hilo.start();
```

2.7.2 Ciclo de Vida y Estados

La Java Virtual Machine (JVM) gestiona los hilos a través de varios estados:

1. **Creado (New):** El objeto existe, pero no se ha llamado a `start()`.

2. **Ejecutable (Runnable):** Listo para correr cuando la JVM lo decida.
3. **En Ejecución (Running):** Usando la CPU actualmente.
 - Puede ceder paso con `yield()`.
4. **Bloqueado (Blocked):** Inactivo esperando un evento (E/S, fin de `sleep`, `notify`, etc.).
5. **Finalizado (Terminated):** Terminó el método `run()`.

INFO

Si haces `hilo.start()` sobre un hilo que ya esta corriendo se lanza una excepción, si quieres lanzar dos veces el mismo hilo, sus referencias han de ser diferentes.

2.7.3 Sincronización (Sección Crítica)

Para evitar conflictos cuando varios hilos acceden a recursos compartidos, Java usa **cerrojos (locks)** mediante la palabra clave `synchronized`. Solo un hilo puede estar dentro de un bloque sincronizado a la vez.

Formas de uso:

1. **Bloque Sincronizado:** Protege un fragmento de código usando un objeto como cerrojo.

```
synchronized(objetoCerrojo) {  
    // Sección crítica  
}
```

2. **Método Sincronizado:** Todo el método es crítico; el cerrojo es el propio objeto (`this`). Por lo que no se puede usar ningún otro método del código mientras esté ocupado este.

```
public synchronized void metodo() { ... }
```

2.7.4 Coordinación de Hilos (Wait y Notify)

Para problemas como el **Productor-Consumidor**, los hilos deben comunicarse. Se usan métodos de la clase `Object` dentro de bloques `synchronized`.

- `wait()`: El hilo suelta el cerrojo y se duerme hasta ser avisado. Si se interrumpe, lanza `InterruptedException`.
- `notify()`: Despierta a un hilo aleatorio que esté esperando en ese objeto.
- `notifyAll()`: Despierta a **todos** los hilos esperando (más seguro para evitar bloqueos indefinidos).

Patrón de uso robusto: Siempre usa `wait()` dentro de un bucle `while` que verifique la condición, no un `if`, para re-comprobar al despertar.

2.7.5 Métodos Útiles de la Clase Thread

- `start()` : Inicia la ejecución del hilo.
- `sleep(long ms)` : Pausa el hilo actual el tiempo indicado.
- `join()` : Hace que el hilo actual espere hasta que el hilo al que se llama termine (ej. el `main` espera a los trabajadores).
- `currentThread()` : Devuelve la referencia al hilo que está ejecutando esa línea de código.

3. Objetos Distribuidos

3.1 Evolución del Paradigma: Del Mensaje al Objeto

Para entender dónde estamos, primero debemos contrastar este nuevo modelo con lo que ya conoces (Sockets/Paso de Mensajes).

3.1.1 Paso de Mensajes (Nivel Bajo)

- **Enfoque:** Orientado a **datos**.
- **Funcionamiento:** Simula una conversación humana. Un proceso envía un mensaje formateado y espera que el otro lo interprete.
- **Acoplamiento:** Alto. Los procesos deben estar activos y sincronizados; si el enlace cae, la colaboración falla.
- **Limitación:** En aplicaciones complejas con miles de interacciones, interpretar mensajes y mantener protocolos se vuelve una tarea "inabordable".

INFO

Orientado a datos es como mandar una carta o formulario. El énfasis está en el **contenido** del mensaje. Los procesos intercambian paquetes de datos con un formato acordado y el receptor debe abrir el paquete, leer los datos y decidir qué hacer con ellos. Es menos intuitivo porque te obliga a pensar en protocolos de comunicación en lugar de en la lógica del programa.

3.1.2 Objetos Distribuidos (Nivel Alto)

- **Enfoque:** Orientado a **acciones**.
- **Abstracción:** Trata los recursos de la red como objetos encapsulados. Los datos pasan a un segundo plano (son meros parámetros).
- **Ventaja:** Es más natural para el desarrollo de software moderno (POO), aunque conceptualmente sea menos intuitivo para los humanos que "enviar un mensaje".

Orientado a acciones es como dar una orden directa. El énfasis está en **invocar una operación** (un verbo, una acción). No piensas en "enviar datos", sino en ejecutar una función específica (método) sobre un objeto, pasando los datos simplemente como argumentos. Aunque ocurre red por debajo, para ti como programador es "pedir que se haga algo".

3.2 Conceptos Fundamentales

Antes de escribir código, debemos definir rigurosamente la terminología arquitectónica.

3.2.1 Tipos de Objetos y Alcance

La distinción fundamental no es física (ordenadores distintos), sino lógica (espacios de memoria distintos).

- **Objeto Local:** Es un objeto convencional. Sus métodos solo pueden ser invocados por referencias que residen en la misma **Máquina Virtual (JVM)**. La llamada es directa a través de la pila de ejecución (stack).
- **Objeto Distribuido (Remoto):** Es un objeto que reside en una JVM distinta (ya sea en el mismo ordenador o en otro conectado por red). Para invocarlo, el cliente debe utilizar mecanismos de red, ya que no tiene acceso directo a su memoria.

3.2.2 La Arquitectura de Intermediarios (Proxies)

Dado que el cliente no puede acceder físicamente a la memoria del servidor, necesitamos una arquitectura que cree la **ilusión de localidad** (transparencia de red).

- **Objeto Servidor (Implementación):** La instancia real de la clase que ejecuta la lógica del negocio. Reside en la memoria (Heap) del servidor.
- **Referencia Remota:** Un identificador único que permite localizar al objeto específico dentro de un servidor específico en la red.
- **RMI Registry (Registro):** Un servicio de nombres (similar a una guía telefónica o DNS) donde el servidor registra (publica) sus objetos remotos asociados a un nombre, y el cliente los busca para obtener su referencia.

****EXPORTAR UN OBJETO** SIGNIFICA INDICAR A LA JVM QUE ESE OBJETO ESTARÁ DISPONIBLE PARA RECIBIR LLAMADAS EXTERNAS. ESTO IMPLICA ABRIR UN PUERTO TCP Y DEJAR UN HILO DE EJECUCIÓN A LA ESCUCHA DE PETICIONES PARA ESE OBJETO.**

El Mecanismo de Stub y Skeleton

La comunicación sigue el patrón *Proxy*. El flujo **real** de una invocación remota es:

1. **Cliente:** Invoca un método sobre el **Stub** (creyendo que es el objeto real).
2. **Stub (Proxy del Cliente):**
 - Realiza el **Marshalling** (serialización): convierte los argumentos del método en un flujo de bytes compatible con la red.
 - Envía la petición a través de la red usando el sistema de transporte.
3. **Red:** Los bytes viajan del cliente al servidor.
4. **Skeleton (Dispatcher del Servidor):**
 - Recibe la petición de red.
 - Realiza el **Unmarshalling** (deserialización): reconstruye los argumentos originales a partir de los bytes recibidos.
 - Identifica qué método se quiere llamar.
5. **Invocación Real:** El Skeleton llama al método en el **Objeto Servidor** real.
6. **Retorno:** El Objeto Servidor devuelve el resultado al Skeleton, quien lo empaqueta (**Marshalling**) y lo envía de vuelta al Stub (que hará el **Unmarshalling** final para el cliente).

NOTA TÉCNICA EN VERSIONES MODERNAS DE JAVA (POST-1.2), LOS `_SKELETONS_` EXPLÍCITOS YA NO SE GENERAN MANUALMENTE. LA JVM UTILIZA ****REFLEXIÓN (REFLECTION)**** Y PROXIES DINÁMICOS PARA MANEJAR ESTE PROCESO, PERO EL CONCEPTO TEÓRICO DE "SKELETON" COMO LA PIEZA QUE DESEMPAQUETA EN EL LADO SERVIDOR SIGUE SIENDO VÁLIDO PARA ENTENDER LA ARQUITECTURA.

ANALOGÍA: EL EMPRESARIO Y EL INTÉRPRETE IMAGINA QUE QUIERES HABLAR CON UN EMPRESARIO JAPONÉS (EL ****OBJETO SERVIDOR****) PERO TÚ SOLO HABLAS ESPAÑOL.

- **El Stub (Tu intérprete):** Está contigo. Tú le hablas en español. Él "empaqueta" (traduce) tu mensaje y lo envía por teléfono. Para ti, hablar con el intérprete es como hablar con el empresario.
- **La Red:** La línea telefónica.
- **El Skeleton (El secretario del empresario):** Está en Japón. Recibe el mensaje, lo "desempaqueta" (traduce al japonés) y se lo dice al empresario. Luego toma la respuesta del empresario, la traduce y te la envía de vuelta.

El **Soporte de Ejecución (Runtime)** sería la compañía telefónica que garantiza que la señal llegue de un lado a otro

3.3 De RPC a RMI: El salto Tecnológico

3.1 Remote Procedure Call (RPC)

RMI no nació de la nada. Su ancestro es **RPC**, popularizado en los años 80.

- En RPC, un proceso llama a un **procedimiento** (función C) en otro proceso
- Utiliza herramientas como `rpcgen` para crear los stubs automáticamente.

3.2 Java RMI (Remote Method Invocation)

RMI es la evolución **Orientada a Objetos de RPC**, exclusiva para el ecosistema Java

- Permite pasar objetos completos como parámetros o valores de retorno, no solo tipos de datos primitivos.
- Utiliza un **Registro RMI** (`rmiregistry`) para la localización de servicios.

3.4 Implementación en Java RMI

Para construir una aplicación distribuida, Java RMI impone una metodología estricta basada en interfaces.

3.4.1 La interfaz Remota

Es el contrato entre cliente y servidor. Define qué se puede hacer, pero no cómo.

- Debe extender `java.rmi.Remote`
- **Regla de Oro:** todos los métodos deben declarar que lanzan `java.rmi.RemoteException`

¿Por qué RemoteException? Porque en sistemas distribuidos, las cosas fallan. La red puede caerse, el servidor puede no existir, o el stub puede perderse. Java obliga al programador a manejar estos fallos explícitamente.

```
public interface SomeInterface extends java.rmi.Remote {  
    public String someMethod1() throws java.rmi.RemoteException;  
}
```

3.4.2 La Implementación del Servidor

Es la clase que hace el trabajo real.

- Debe implementar la interfaz remota definida anteriormente
- Generalmente hereda de `UnicastRemoteObject`. Esto facilita que el objeto sea "exportable" a la red.
- Debe tener un constructor que lance `RemoteException` (porque la clase padre lo hace).

3.4.3 Generación de Stubs y Skeletons

Una vez compiladas las clases, se usa el compilador RMI para generar los proxies:

- Comando: `rmic NOMBREDeLaImplementacion`
- Resultado: Generado dos archivos: `_Stub.class` y `_Skel.class`
 - **Importante:** El archivo `_Stub.class` debe copiarse manualmente a la máquina del cliente (o descargarse dinámicamente) para que este sepa cómo comunicarse.

INFO

A día de hoy se generan de forma automática a no ser que uses una versión inferior a java 5 (que debe ser la puta mierda que usa el profesor, el pavo no actualiza su asignatura desde 2008).

3.5 El Registro y la Puesta en Marcha

3.5.1 El RMI Registry

Es un servicio simple que se ejecuta por defecto en el puerto TCP 1099. Actúa como un mapa de `String` → `Objeto`.

- Se puede iniciar desde consola: `rmiregistry`
- O desde código Java: `LocateRegistry.createRegistry(puerto)`

3.5.2 Lógica del Servidor (Publicación)

El servidor debe instanciar el objeto y "publicarlo" usando la clase `Naming`.

- **URL RMI:** `rmi://host:port/nombre_servicio`.
- **Métodos Clave:**
 - `bind(url, objeto)`: registra el objeto. Falla si ya existe ese nombre
 - `rebind(url, objeto)`: registra o sobrescribe si ya existe (más seguro/común)

3.5.3 Lógica del Cliente (Búsqueda)

El cliente necesita localizar el objeto para obtener una referencia.

- Construye la URL de búsqueda
- Usa `Naming.lookup(url)`.

Advertencia Crítica: `Naming.lookup` devuelve un objeto tipo `Remote`. Debes hacer un **cast** a la **Interfaz Remota**, NUNCA a la clase de implementación del servidor. El cliente no conoce la implementación, solo la interfaz.

3.6 Ejemplo Completo

Paso 1: La Interfaz Remota

Define el contrato. Debe extender `Remote` y lanzar `RemoteException`.

```
import java.rmi.Remote;
import java.rmi.RemoteException;

// Interfaz compartida por Cliente y Servidor
public interface Calculadora extends Remote {
    // Método que queremos invocar remotamente
    public int sumar(int a, int b) throws RemoteException;
}
```

Paso 2: El Servidor (Implementación y Publicación)

Implementa la lógica y se registra en el `rmiregistry`.

```
import java.rmi.server.UnicastRemoteObject;
import java.rmi.Naming;
import java.rmi.RemoteException;
import java.rmi.registry.LocateRegistry;

// La implementación real
public class ServidorCalculadora extends UnicastRemoteObject implements Calculadora {

    protected ServidorCalculadora() throws RemoteException {
        super(); // Necesario para exportar el objeto
    }

    @Override
    public int sumar(int a, int b) throws RemoteException {
```

```

        System.out.println("Me han pedido sumar: " + a + " + " + b);
        return a + b;
    }

    public static void main(String[] args) {
        try {
            // 1. Iniciar el registro RMI en el puerto 1099 (para no usar consola)
            LocateRegistry.createRegistry(1099);

            // 2. Crear el objeto
            ServidorCalculadora objetoRemoto = new ServidorCalculadora();

            // 3. Registrarlo con un nombre ("Naming.rebind")
            Naming.rebind("rmi://localhost:1099/MiCalculadora", objetoRemoto);

            System.out.println("Servidor listo y esperando...");
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}

```

Paso 3: El Cliente

Busca el objeto y lo usa.

```

import java.rmi.Naming;

public class ClienteCalculadora {
    public static void main(String[] args) {
        try {
            // 1. Buscar el objeto en el registro (Lookup)
            // OJO: Hacemos casting a la INTERFAZ, no a la clase del servidor
            Calculadora calc = (Calculadora)
Naming.lookup("rmi://localhost:1099/MiCalculadora");

            // 2. Usar el objeto como si fuera local
            int resultado = calc.sumar(5, 10);

            System.out.println("El resultado de la suma remota es: " + resultado);
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}

```

```
}  
}  
}
```

3.7 Algoritmo de Desarrollo Paso a Paso

Para no perderte en la práctica, sigue este "algoritmo" riguroso extraído del profesor (gilipollez histórica, programa con lógica y pista, no vas a hacerte el código sin haber hecho antes la interfaz):

1. **Definir Interfaz:** Crear `Interfaz.java` (extiende `Remote`) y compilar.
2. **Implementar:** Crear `Impl.java` (implementa `Interfaz`, extiende `UnicastRemoteObject`) y compilar.
3. **Generar Proxies:** Ejecutar `rmic Impl`. Obtendrás el Stub y el Skeleton.
4. **Desarrollar Servidor:** Crear `Server.java` que instancia `Impl` y hace `Naming.rebind`.
5. **Desarrollar Cliente:** Crear `Client.java` que hace `Naming.lookup` y usa la interfaz.
6. **Despliegue de Ficheros:**
 - **Servidor:** Necesita `Interfaz.class`, `Impl.class`, `Server.class`, `Skeleton.class`.
 - **Cliente:** Necesita `Interfaz.class`, `Client.class`, `Stub.class`.

IMPORTANT

Java RMI es por naturaleza multihilo. Cuando recibe una petición de un cliente, por ejemplo si un cliente *A* invoca a un método remoto *A*, el entorno de ejecución de Java RMI creará un hilo para ejecutar ese método remoto. Si al mismo tiempo otro cliente *B* le manda otra petición, creará otro hilo. Ambos hilos pueden ejecutar el mismo código del objeto remoto **en paralelo**.

Para que funcione **todo el código debe ser Thread Safe**.

4. RMI Avanzado

INFO

RMI avanzado, menudo puto fenómeno, era mejor que lo llamase RMI retrasado. La mayoría de cosas de este tema a día de hoy están obsoletas y no se usan. Java RMI tampoco se suele usar mucho.

Era Clásica (la puta mierda de la que hablan las diapositivas)

Esta es la metodología original (JDK 1.1 - 1.4) y la que se detalla explícitamente en el material de estudio.

- **¿Qué se usaba?** Se utilizaban **Archivos Físicos Generados**. Era obligatorio pre-compilar clases auxiliares específicas para cada objeto remoto.
- **El Proceso Manual:**
 1. Escribías tu implementación del servidor (ej. `SomeImpl.java`).
 2. Ejecutabas una herramienta de línea de comandos llamada `rmic` (RMI Compiler) contra tu clase compilada.
 3. Esta herramienta generaba dos nuevos archivos `.class` en tu disco duro:
 - `SomeImpl_Stub.class` (Para el cliente).
 - `SomeImpl_Skel.class` (Para el servidor).
- **¿Cómo funcionaba?**
 - **En el Cliente:** Necesitabas tener físicamente el archivo `_Stub.class` (copiado a mano o descargado vía web). El cliente invocaba métodos sobre este objeto físico.
 - **En el Servidor:** Existía un objeto **Skeleton** real interpuesto. El flujo era rígido: `Red -> Objeto Skeleton -> Objeto Real`. El Skeleton era el único que sabía cómo desempaquetar los datos para esa clase específica.

La Era Moderna (esto no entra)

Con la evolución del lenguaje, Java eliminó la necesidad de generar código "sucio" intermedio, haciendo el proceso transparente.

- **¿Qué se usa ahora?** Se usan **Proxies Dinámicos** y **Reflexión**. Los archivos físicos `_Stub` y `_Skel` han desaparecido del flujo de trabajo diario.
- **El Proceso Automático:**

1. Escribes tu implementación.
2. Lanzas el servidor. **No ejecutas** `rmic`.
3. Java se encarga del resto en tiempo de ejecución.

- **¿Cómo funciona ahora?**

- **Adiós al Skeleton:** El servidor RMI moderno es genérico. Ya no necesita una clase `_Skel` intermedia. Usa la **Reflexión** de Java para inspeccionar el objeto real y llamar al método adecuado directamente al recibir la petición de red.
- **El Stub Fantasma:** Cuando el cliente busca el objeto (`lookup`), la Máquina Virtual detecta que es una interfaz remota y **genera el código del Stub en la memoria RAM** al instante (Proxy Dinámico). El cliente cree que usa una clase compilada, pero es un objeto sintético creado "al vuelo".

4.1 Distribución Dinámica de Código: Stub Downloading

En el modelo básico que vimos en el tema anterior, tenías que copiar manualmente el archivo `Stub.class` del servidor. Esto es un problema de mantenimiento: si cambias el servidor, tienes que redistribuir el stub a todos los clientes.

4.1.1 El Concepto

Imagina que quieres pedir comida a un restaurante (el Servidor). Para saber qué pedir, necesitas su carta o menú (el Stub/Interfaz).

- **Sin Stub Downloading (Forma manual):** Tú tienes una copia impresa del menú en tu casa. Si el restaurante cambia sus platos o precios, tu menú de papel queda obsoleto. Llamarás pidiendo algo que ya no existe o con el precio equivocado. Para arreglarlo, tendrías que ir físicamente al restaurante, recoger el nuevo menú y volver a casa.
 - *En Informática:* Esto equivale a tener que copiar manualmente el archivo `Stub.class` en cada ordenador de cada cliente cada vez que actualizas el código del servidor. Si tienes 100 clientes, es inviable.
- **Con Stub Downloading (Forma dinámica):** Tú no guardas el menú. Cuando quieres pedir, entras en la web del restaurante y ves el menú actual en ese preciso instante. Si el restaurante cambia el menú, tú ves el cambio automáticamente la próxima vez que entres.
 - *En Informática:* El cliente no necesita tener el archivo `Stub.class` instalado en su disco duro. Cuando se conecta, el sistema le dice: "Oye, para hablar conmigo necesitas este código, descárgalo de aquí ahora mismo".

4.1.2 Implementación Técnica

Para que esto funcione, el servidor RMI debe decirle al cliente "donde buscar" las clases que no tiene. Esto se hace mediante propiedades del sistema al lanzar el servidor:

- **Codebase:** Indica la URL donde están los stubs. `java -Djava.rmi.server.codebase=http://www.miservidor.com/clases/`
- **Hostname:** la IP pública o nombre del servidor. `-Djava.rmi.server.hostname=mi.ip publica`
- **Policy:** archivo de permisos de seguridad. `-Djava.security.policy=java.policy`

Funcionamiento

- **El Servidor se prepara:** Cuando arrancas el servidor, le indicas una propiedad llamada `codebase`. Le estás diciendo a RMI: "Si alguien te pide el Stub y no lo tiene, dile que lo descargue de esta dirección web" (por ejemplo, un servidor HTTP).
- **El Cliente busca:** El cliente llama al Registro RMI buscando el objeto (`lookup`).
- **Detección de falta:** Si el cliente comprueba que **no tiene** la clase del Stub en su disco duro local...
- **Descarga automática:** ...automáticamente usa la URL que proporcionó el servidor (el `codebase`) para descargar el archivo `.class` a través de HTTP, cargarlo en memoria y usarlo.

4.2 El Gestor de Seguridad (Security Manager)

Al permitir `Stub Downloading`, tu programa cliente está descargando y ejecutando código de una red externa. Esto es un riesgo de seguridad enorme.

4.2.1 RMISecurityManager

Es una clase que restringe lo que el código descargado puede hacer.

- Es **obligatorio** instanciarlo tanto en el cliente como en el servidor si vas a usar carga dinámica de clases.
- Se activa en el código con: `System.setSecurityManager(new RMISecurityManager());`

4.2.2 El Fichero de Políticas (`java.policy`)

El Security Manager es un "portero" que no deja pasar a nadie a menos que esté en la lista de invitados. Esa lista el fichero `java.policy`.

Debes conceder permisos explícitos:

- **Conectar al Registry:** Permiso para hablar al puerto 1099
- **Descargar Stubs:** Permiso para conectar al puerto 80 (HTTP)

- **Conexiones efímeras:** Permiso para que puertos dinámicos (' 1024–65535) usados en la comunicación de vuelta.

```
// archivo: cliente.policy
grant {
    // 1. Permiso para hablar con el Registry (puerto 1099 normalmente)
    // "connect": poder llamar. "resolve": poder buscar la IP.
    permission java.net.SocketPermission "localhost:1099", "connect, resolve";

    // 2. Permiso para conexiones efímeras (puertos dinámicos que usa RMI para
    responder)
    // *:1024–65535 significa "cualquier IP" en el rango de puertos del 1024 al
    final.
    // "accept": permite recibir llamadas de vuelta del servidor.
    permission java.net.SocketPermission " *:1024-", "connect, accept, resolve";

    // 3. Si descargas código de un servidor web (ej. puerto 80)
    permission java.net.SocketPermission "miservidorweb.com:80", "connect";
};
```

Por buenas prácticas, se recomienda usar gestores de seguridad en **todas** las aplicaciones RMI, incluso si no usas descarga dinámica.

El fichero `.policy` **no se aplica a "un objeto" (como el Stub), se aplica a toda la Máquina Virtual Java (JVM)** que está ejecutando ese programa.

Cuando activas el `RMISecurityManager`, pones a **toda la aplicación** en un estado de "cuarentena" o "libertad condicional". Por defecto, Java asume que **nadie** (ni el código descargado, ni tu propio código del servidor) tiene permiso para hacer operaciones de red.

La colocación de archivos si usamos **Stub downloading y security manager** sería: - **Servidor:**

Necesita `Interfaz.class`, `Impl.class`, `Server.class`, `Skeleton.class`, `Stub.class`, `java.policy`. - **Servidor HTTP:** `Stub.class` - **Cliente:** Necesita `Interfaz.class`, `Client.class`, `java.policy`.

A. Policy del Cliente (Foco: Salir)

El cliente necesita permiso para **ir** al servidor.

```
grant {
    // Permiso para CONECTAR (connect) con el servidor
    permission java.net.SocketPermission "ip_del_servidor:1024-", "connect, resolve";
    permission java.net.SocketPermission "ip_del_servidor:1099", "connect, resolve";
    permission java.net.SocketPermission "ip_del_servidor_web:80", "connect,
```

```
resolve";  
};
```

B. Policy del Servidor (Foco: Entrar y Registrarse)

El servidor necesita permiso para **recibir** visitas y **registrarse**.

```
grant {  
    // Permiso para ACEPTAR (accept/listen) conexiones de cualquiera  
    permission java.net.SocketPermission "*:1024-", "accept, listen, connect,  
resolve";  
  
    // Permiso para hablar con el Registry (connect)  
    permission java.net.SocketPermission "localhost:1099", "connect, resolve";  
};
```

4.3 Comunicación Bidireccional: RMI Callbacks

Hasta ahora, el modelo era **Pull** (Cliente pide → Servidor Responde). El servidor era pasivo. Pero, ¿y si el servidor necesita avisar al cliente de algo (ej: una subasta ha terminado, un chat, monitorización)?

4.3.1 El Problema del Polling

Sin callbacks, el cliente tendría que preguntar constantemente: "¿Ya terminó? ¿Ya terminó?". Esto se llama **Polling** y es ineficiente y satura la red.

4.3.2 La Solución: Callback (Llamada inversa)

El cliente se "registra" en el servidor y espera a que el servidor le llame.

Esto convierte la arquitectura en **Duplex**: ambos lados actúan como cliente y servidor en momentos diferentes.

4.3.2 Estructura de un Callback

Para implementar esto, necesitamos **dos interfaces remotas** y **dos juegos de Stubs/Skeletons**.

Interfaz del Servidor: Define un método para que el cliente se suscriba.

- Ejemplo: `addCallback(HelloCallbackInterface cliente)`.

Interfaz del Cliente (CallbackInterface): Define el método que el servidor invocará.

- Ejemplo: `public void callMe(String msg)`.

La colocación de archivos con RMI Callback es:

- **Servidor:** `IntServer.class`, `ImplServer.class`, `Server.class`, `IntClient.class`, `StubClient.class`, `SkellServer.class`.
- **Cliente:** `IntClient.class`, `ImplCliente.class`, `Client.class`, `IntServer.class`, `StubServer.class`, `SkellClient.class`.

4.3.4 Flujo de Ejecución

- **Cliente:** crea un objeto que implementa su interfaz de callback y lo **exporta**.
- **Registro:** el cliente llama al servidor (`addCallback`) pasándole una referencia a sí mismo (`this`)
- **Servidor:** guarda al cliente en una lista (ej: un `ArrayList`)
- **Notificación:** cuando ocurre el evento, el servidor recorre la lista e invoca el método `callMe` de cada cliente.

4.3.5 Ejemplo

Interfaz del Callback (Lo que tiene el Cliente) Define el método que el servidor llamará para notificar.

```
import java.rmi.Remote;
import java.rmi.RemoteException;

// Interfaz que implementará el Cliente
public interface InterfaceCallback extends Remote {
    // El servidor usará este método para "gritarle" al cliente
    public void notificar(String mensaje) throws RemoteException;
}
```

Interfaz del Servidor (El servicio principal) Define el método para registrarse.

```
import java.rmi.Remote;
import java.rmi.RemoteException;

public interface InterfaceServidor extends Remote {
    // Método clásico del servicio
    public String decirHola() throws RemoteException;

    // MÉTODO CLAVE: El cliente pasa una referencia a sí mismo (su interfaz)
    public void registrarCliente(InterfaceCallback cliente) throws RemoteException;
}
```

El Servidor (Implementación). El servidor debe mantener una lista de los clientes conectados para poder avisarles luego.

```
import java.rmi.server.UnicastRemoteObject;
import java.rmi.RemoteException;
import java.rmi.Naming;
import java.rmi.registry.LocateRegistry;
import java.util.Vector; // Usado en el PDF

public class ServidorImpl extends UnicastRemoteObject implements InterfaceServidor {

    // Lista para guardar las referencias a los clientes
    private Vector<InterfaceCallback> listaClientes;

    public ServidorImpl() throws RemoteException {
        super();
        listaClientes = new Vector<>();
    }

    @Override
    public String decirHola() throws RemoteException {
        return "Hola desde el Servidor RMI";
    }

    @Override
    public void registrarCliente(InterfaceCallback cliente) throws RemoteException {
        // Guardamos al cliente en la lista
        System.out.println("Servidor: Nuevo cliente registrado.");
        listaClientes.add(cliente);
    }

    // Método interno para simular un evento y avisar a todos
    public void realizarCallback() {
        System.out.println("Servidor: Iniciando callbacks a " + listaClientes.size()
+ " clientes...");

        // Recorremos la lista e invocamos al cliente [cite: 287]
        for (InterfaceCallback cliente : listaClientes) {
            try {
                // AQUÍ OCURRE LA MAGIA: El servidor llama al cliente
                cliente.notificar("¡El evento ha ocurrido! Saludos del servidor.");
            } catch (RemoteException e) {
                System.out.println("Error al contactar con un cliente: " +
```

```

e.getMessage());
    }
}

public static void main(String[] args) {
    try {
        LocateRegistry.createRegistry(1099);
        ServidorImpl servidor = new ServidorImpl();
        Naming.rebind("rmi://localhost/ServicioCallback", servidor);
        System.out.println("Servidor listo.");

        // Simulamos que pasados 5 segundos, ocurre el evento
        Thread.sleep(5000);
        servidor.realizarCallback();

    } catch (Exception e) {
        e.printStackTrace();
    }
}
}

```

El Cliente (Implementación y Registro). Aquí está la parte crítica: el cliente debe implementar `InterfaceCallback` y **exportarse** para que el servidor pueda llamarle.

```

import java.rmi.server.UnicastRemoteObject;
import java.rmi.RemoteException;
import java.rmi.Naming;

// El cliente DEBE implementar la interfaz de callback [cite: 241]
public class ClienteConCallback extends UnicastRemoteObject implements
InterfaceCallback {

    public ClienteConCallback() throws RemoteException {
        super(); // Esto exporta al cliente automáticamente como objeto remoto
    }

    // Este es el método que ejecutará el servidor remotamente
    @Override
    public void notificar(String mensaje) throws RemoteException {
        System.out.println("CLIENTE RECIBIÓ NOTIFICACIÓN: " + mensaje);
    }
}

```

```

public static void main(String[] args) {
    try {
        // 1. Instanciar el cliente (se exporta al nacer por heredar de
UnicastRemoteObject)
        ClienteConCallback miCliente = new ClienteConCallback();

        // 2. Buscar al servidor
        String url = "rmi://localhost/ServicioCallback";
        InterfaceServidor servidor = (InterfaceServidor) Naming.lookup(url);

        // 3. Registrarse pasando 'this' (nuestra propia referencia remota)
[cite: 334]
        System.out.println("Cliente: Registrándome para callbacks...");
        servidor.registrarCliente(miCliente);

        System.out.println("Cliente: Esperando evento...");

        // Mantenemos el cliente vivo para recibir la llamada
        // (En un caso real, la aplicación seguiría ejecutándose)

    } catch (Exception e) {
        e.printStackTrace();
    }
}
}

```

4.4 Serialización y Paso de Objetos

A veces no basta con pasar números o texto (`int`, `String`). Necesitamos pasar estructuras de datos complejas u objetos propios.

4.4.1 Concepto de Serialización

Es el mecanismo para convertir un objeto (su código y sus datos) en una secuencia de bytes (stream) para enviarlo por la red y reconstruirlo idéntico en el otro lado.

- Java garantiza que el objeto reconstruido en el servidor funcionará igual que en el cliente.

4.4.2 Requisito Fundamental

Para pasar un objeto como argumento en RMI, la clase de ese objeto **debe implementar la interfaz** `java.io.Serializable`. Si no lo hace, RMI lanzará una excepción al intentar enviarlo.

Todos los atributos dentro del objeto también deben ser serializables (primitivos, Strings, o referencias a otros objetos Serializables).

Debe tener un constructor sin parámetros, necesario para que RMI pueda reconstruir el objeto destino mediante reflexión.

Si hay datos que no quieres enviar (ej: una contraseña o una conexión a BD abierta), debes marcarlos con la palabra clave `transient`.

```
import java.io.Serializable;

// 1. Implementa la interfaz (La etiqueta)
public class Empleado implements Serializable {

    private String nombre;

    // 3. Recursividad: La clase 'Direccion' TAMBIÉN debe ser Serializable
    // Si 'Direccion' no lo fuera, daría error al enviar 'Empleado'.
    private Direccion domicilio;

    // 4. Transient: Este dato NO viaja. En el servidor llegará como null.
    // Útil para seguridad o datos que solo tienen sentido localmente.
    private transient String claveSecreta;

    // 2. Constructor vacío: Necesario para reconstruir el objeto al llegar
    public Empleado() {
    }

    // Constructor normal para usarlo tú
    public Empleado(String nombre, Direccion d) {
        this.nombre = nombre;
        this.domicilio = d;
    }
}
```

5. Redes P2P

INFO

Muy loco que el colega te chante una presentación en inglés joseada de otra universidad

5.1 El Problema: Arquitectura Cliente-Servidor vs P2P

Para entender P2P, primero debemos mirar que lo que queremos sustituir: el modelo tradicional **Cliente-Servidor**.

- **Modelo Tradicional:** Un servidor potente y fiable centraliza los datos. Los clientes solo piden información
- **Limitaciones:** El servidor es un cuello de botella, un punto único de fallo y es costoso de escalar. Además, los recursos de los clientes (tu ordenador) están desaprovechados.

La computación P2P es el intercambio directo de recursos (procesamiento, almacenamiento, ancho de banda) entre sistemas.

- **Democracia:** todos los nodos son a la vez **clientes y servidores**
- **Autonomía:** no hay una autoridad central ; los nodos entran y salen de la red dinámicamente.

Beneficio Clave: La **Escalabilidad**. Cuantos más usuarios (consumidores) entran, más recursos (donantes) hay disponibles. El sistema crece orgánicamente con la demanda .

5.2 Primera Generación: Sistemas P2P No Estructurados

Llamamos "no estructurados" a aquellos sistemas donde no hay una regla matemática estricta sobre dónde se guarda cada archivo. La búsqueda es algo caótica o centralizada.

5.2.1 Napster (El origen centralizado)

Aunque revolucionó el intercambio de música, Napster no era P2P puro.

- **Arquitectura:** Híbrida.
 - **Búsqueda:** Centralizada. Tú enviabas tu consulta a un servidor central de Napster ("¿Quién tiene la canción X?").

- **Descarga:** P2P. Una vez tienes la IP del usuario que tiene el archivo, te conectas directamente a él para bajarlo.
- **Problema:** El servidor central es un **punto único de fallo**, un cuello de botella y un blanco fácil para demandas legales

5.2.2 Gnutella (Descentralización total)

Para evitar los problemas legales de Napster, Gnutella eliminó el servidor central.

- **Funcionamiento:** Búsqueda por **Inundación (Flooding)**.
 - Preguntas a tus vecinos: "¿Tenéis este archivo?".
 - Ellos preguntan a sus vecinos, y así sucesivamente.
- **Control:** Para que el mensaje no viva eternamente, se usa un **TTL** (Time To Live) que elimina la petición tras unos cuantos saltos (hops).
- **Problema:** Genera muchísimo tráfico de red y no garantiza encontrar el archivo (si está más lejos que el TTL, no lo ves)

5.2.3 Kazaa/FastTrack (El modelo Híbrido)

Aprendiendo de los dos anteriores, Kazaa introdujo el concepto de **Super-nodos (Super-peers)**.

- **Jerarquía:** No todos los nodos son iguales. Los ordenadores potentes y con buena conexión se convierten automáticamente en "Super-peers".
- **Funcionamiento:** Actúan como pequeños servidores Napster locales. Los nodos normales suben sus listas de archivos al Super-peer y le preguntan a él.

5.2.4 El Problema del "Free Riding" (El gorrón)

Un desafío social de estas redes es que requieren altruismo. Estudios en Gnutella mostraron que el **63% de los usuarios nunca compartían nada**, solo descargaban. El 15% de los usuarios aportaba el 94% del contenido.

5.2.5 Freenet

En las generaciones anteriores (Napster, Gnutella, Kazaa), la identidad era transparente: los usuarios sabían de quién descargaban y los demás sabían quién enviaba una consulta. Esto generaba problemas de privacidad y legales

Freenet surge con un objetivo distinto: **el Anonimato**.

- **Funcionamiento ("Consultas Inteligentes"):**

- Utiliza un mecanismo de **descubrimiento incremental**.
- Lo más ingenioso es cómo fluyen los datos: La respuesta viaja siguiendo el **camino inverso** de la consulta.

- **La Ventaja del Anonimato:**

- Es imposible saber si un nodo está **iniciando** una petición o simplemente **reenviando** la de otro.
- Es imposible saber si un nodo está **consumiendo** (leyendo) el dato o simplemente **reenviándolo**.
- Esto crea una "negación plausible" para los usuarios de la red.

5.3 Segunda Generación: Sistemas Estructurados (DHTs)

Las redes anteriores (Napster, Gnutella) no garantizan que encuentres un archivo aunque exista, o son ineficientes. La academia (universidades/investigación) propuso una solución matemática: las **Tablas Hash Distribuidas (DHT)**.

5.3.1 ¿Qué es una DHT?

Una **Tabla Hash Distribuida** es una clase de sistema distribuido descentralizado que ofrece un servicio de búsqueda (lookup) similar a una tabla hash clásica (pares `clave` → `valor`), pero con una diferencia fundamental:

- **Distribución:** Los datos (pares clave-valor) no están en un solo ordenador. Están **repartidos** entre todos los nodos participantes de la red.
- **Responsabilidad:** Cada nodo es responsable de un pequeño rango de claves.
- **Enrutamiento:** Cualquier nodo puede encontrar eficientemente ($O(\log N)$) qué otro nodo tiene el dato que busca, sin necesidad de un servidor central.
- **Interfaz Genérica:** Solo necesitas dos operaciones:
 - `put(key, value)`: Guarda algo.
 - `value = get(key)`: Recupera algo.

A continuación, los tres algoritmos principales de DHT

5.3.2 Chord (El anillo)

Imagina una mesa redonda gigante o un reloj.

El Círculo Identificador (The Identifier Circle):

- **La Mesa:** Imagina que todos los ordenadores (Nodos) y todos los archivos (Claves) tienen un número de identificación (ID) único (por ejemplo, del 0 al 100).
- **Organización:** Los nodos se sientan alrededor de la mesa ordenados por su número (el nodo 5, luego el 20, luego el 38...).
- **La Regla de Oro (Sucesor):** ¿Quién guarda la llave del archivo número 54? No se la damos al nodo 54 (porque igual no existe), sino al **primer nodo que encontremos igual o mayor que 54** siguiendo las agujas del reloj.
 - *Ejemplo:* La clave **K54** se guarda en el nodo **N56** porque es su "sucesor" inmediato en el círculo .

La Tabla de Dedos (Finger Table) - El Atajo: Si el nodo 1 quiere hablar con el nodo 50, podría pasar el mensaje al vecino de al lado (nodo 2), y este al 3... pero tardaría mucho ($O(N)$). Para evitar esto, Chord usa "dedos" mágicos.

- **Los Saltos:** Cada nodo tiene una pequeña agenda (Finger Table) con unos pocos contactos.
- **Matemática de los saltos:** En lugar de conocer a sus vecinos inmediatos, conoce a nodos a distancias de potencia de 2 ($2^0, 2^1, 2^2, 2^4 \dots$).
 - El dedo 1 apunta a alguien a distancia 1.
 - El dedo 2 apunta a alguien a distancia 2.
 - El dedo 3 apunta a alguien a distancia 4, luego 8, 16, 32....
- **El Resultado:** Con muy poca información (tabla pequeña de tamaño $O(\log N)$) puedes cruzar el círculo entero dando saltos gigantes. Cada salto te acerca "a la mitad" del camino hacia tu destino.

El Problema de la Localidad (Network Locality) Aquí viene la crítica de Chord. El anillo es **lógico**, no físico.

- Imagina que en el anillo, el Nodo 20 es vecino del Nodo 40.
- Pero en la vida real, el Nodo 20 está en Oregón (EE.UU.) y el Nodo 40 está en Utah (EE.UU.), pero para llegar de uno a otro, el algoritmo te hace pasar por el Nodo 80 que está en California o el 41 que está en Nueva York .
- **Conclusión:** Tus vecinos en el anillo pueden estar en la otra punta del mundo, haciendo que la conexión sea lenta aunque el algoritmo sea rápido.

5.3.3 Pastry (El código postal)

Pastry intenta arreglar el problema de que "mi vecino lógico está en otro continente".

- **La Idea:** Funciona como los números de teléfono o códigos postales.
- **El Enrutamiento:** Si yo soy el nodo **65a1fc** y busco al **d46a1c** , intento enviar el mensaje a alguien que se parezca un poco más al destino.

1. Primero busco a alguien que empiece por `d...`
2. Luego a alguien que empiece por `d4...`
3. Luego `d46...`.

- **La Mejora (Localidad):** Si necesito pasar el mensaje a alguien que empiece por `d...` y conozco a tres nodos que empiezan por `d`, Pastry elige **el que esté físicamente más cerca de mí** (menor latencia).
- **Diferencia con Chord:** Mientras Chord te obliga a saltar a un punto fijo del anillo (aunque esté en China), Pastry te deja elegir entre varios candidatos para dar el siguiente paso, permitiendo optimizar la velocidad real.

5.3.4 CAN (El mapa de coordenadas)

Olvídate de anillos y códigos. Piensa en un mapa gigante (un plano cartesiano de coordenadas X e Y).

- **El Terreno:** Todo el espacio posible (el mapa) se divide en rectángulos (zonas).
- **Los Dueños:** Cada nodo es dueño de un rectángulo específico.
- **Guardar datos:** Si quieres guardar un archivo, haces una fórmula matemática (hash) que te da unas coordenadas, por ejemplo `(x=0.8, y=0.3)`. El archivo se guarda en el ordenador que sea "dueño" del rectángulo donde cae ese punto.
- **Enrutamiento:** Si yo estoy en la zona `(0,0)` y quiero enviar algo a `(1,1)`, simplemente le paso el mensaje a mi vecino de la derecha o de arriba, acercándome geoméricamente al destino.

NOTA

Yo ns pa que manda estudiar estos algoritmos, nunca los preguntó en el final. Yo creo que simplemente le dio pereza recortar la presentación que robó a otra universidad. Por norma general los profesores consideran que su tiempo vale más que el nuestro.

6. Arquitecturas Orientadas a Servicios y Servicios Web

NOTA

Yo este tema no se para que lo damos, a día de hoy tengo entendido que se usa REST con JSON en vez de SOAP con XML. Creo que estaban mirando de cambiar el contenido de la asignatura así que no tengo claro que el tipo vaya a cambiar el examen y preguntar algo de esto. Pero más vale prevenir que curar.

6.1 El Desafío de la Comunicación en Entornos Web Abiertos

El problema fundamental de la computación distribuida a escala global radica en la necesidad de invocar métodos de objetos situados en máquinas remotas a través de redes no confiables como Internet. El objetivo es lograr que una aplicación cliente local pueda ejecutar lógica de negocio alojada en un servidor remoto (posiblemente ubicado en otro continente) con la misma transparencia que si fuera una llamada local.

6.1.1 Antecedentes y Limitaciones: La Era de RPC, CORBA y DCOM

Inicialmente, se utilizaron protocolos binarios que establecían conexiones directas mediante puertos específicos. Este enfoque fracasó en la web abierta por dos factores críticos:

1. **Seguridad Perimetral:** Los cortafuegos corporativos bloquean por defecto cualquier tráfico en puertos no estándar, impidiendo la comunicación externa.
2. **Acoplamiento Tecnológico:** Estas arquitecturas exigían homogeneidad, requiriendo que cliente y servidor operasen bajo la misma plataforma o lenguaje, lo que limitaba la interoperabilidad.

6.1.2 La Aproximación Basada en HTTP: El Uso de Java Servlets

Para superar las restricciones de seguridad, la ingeniería de software evolucionó hacia el aprovechamiento de la infraestructura existente de la Web. Se identificó que el puerto 80 (HTTP), utilizado para la navegación web estándar, permanecía abierto en casi todos los cortafuegos.

Esta estrategia dio lugar al concepto de **Tunneling** a través de Java Servlets. La técnica consistía en encapsular las peticiones de datos dentro de tráfico HTTP estándar, permitiendo que la información

atravesara los cortafuegos sin ser bloqueada. Los Servlets actuaban como pequeños programas en el servidor que escuchaban estas peticiones en el puerto 80.

Aunque resolvió el problema de conectividad, introdujo una deficiencia estructural: los Servlets devolvían contenido en HTML (diseñado para presentación visual), obligando al cliente a realizar un *parsing* complejo para extraer los datos. Esto generaba un sistema frágil y difícil de mantener, donde cualquier cambio en la interfaz visual rompía la lógica de comunicación.

6.1.3 La Evolución hacia la Arquitectura Orientada a Servicios (SOA)

Como respuesta a la necesidad de una comunicación robusta, estandarizada y capaz de atravesar cortafuegos, surgió la **Arquitectura Orientada a Servicios (SOA)**. Esta arquitectura propuso una solución definitiva al combinar la ubicuidad del transporte web con un formato de datos estricto.

La ecuación fundamental de SOA en el contexto de los Servicios Web se define por la integración de dos estándares: el **Protocolo HTTP** como medio de transporte universal para garantizar la conectividad, y el lenguaje **XML** para la representación de datos, asegurando que la información sea estructurada, legible por máquinas e independiente de la plataforma de presentación.

6.2 Arquitectura SOA

SOA (Arquitectura Orientada a Servicios): Es un estilo arquitectónico para construir sistemas distribuidos basado en la integración de servicios autónomos, débilmente acoplados e interoperables. Permite que aplicaciones desarrolladas en tecnologías heterogéneas se comuniquen mediante estándares abiertos, fundamentalmente combinando un transporte universal (**HTTP**) con un formato de datos estructurado (**XML**).

La arquitectura se basa en tres actores y cuatro estándares fundamentales:

Actor	Acción	Estándar Clave	Función del Estándar
Proveedor	Crea y expone	WSDL	Descripción: Fichero XML que define <i>qué</i> hace el servicio (interfaz) y <i>cómo</i> llamarlo.
Registro	Directorio ("Páginas Amarillas")	UDDI	Publicación: Donde se registran los servicios para ser encontrados. (<i>Nota: Baja implantación real</i>).
Consumidor	Busca y usa	SOAP	Mensajería: Formato del sobre XML para enviar la petición y recibir la respuesta.
(Todos)	Canal de comunicación	HTTP	Transporte: La carretera por la que viajan los mensajes SOAP.

Flujo de Funcionamiento (Publish - Find - Bind):

1. **Despliegue (Provider):** El proveedor crea el servicio y su contrato (**WSDL**) accesible vía URL.

2. **Publicación (Provider → UDDI):** Registra el servicio en el directorio UDDI para que sea visible.
3. **Descubrimiento (Consumer → UDDI):** El consumidor busca en UDDI y obtiene la URL del **WSDL**.
4. **Generación (Consumer):** Con el WSDL, el consumidor genera automáticamente los **Stubs/Proxies** (clases locales para comunicar).
5. **Invocación (Consumer → Provider):** Se lanza la petición encapsulada en **SOAP** a través de **HTTP**.

Gracias al uso de **XML**, se pueden integrar aplicaciones dispares:

- **Lenguajes diferentes:** Se traduce de XML al lenguaje local (Java, Python, C#, etc.).
- **Modelos de datos diferentes:** Se traduce el modelo del formato XML al modelo interno de la aplicación.

6.3 El Manual de Instrucciones: WSDL

Para conectar tu código con un servicio remoto, necesitas un contrato: el **WSDL** (*Web Service Description Language*). Es un XML jerárquico dividido en dos bloques:

6.3.1 Descripción Abstracta (La Lógica - El "Qué")

Define la interfaz funcional sin importar el lenguaje de programación.

1. **types (Diccionario):** Define los datos (`int`, `Cliente`) usando **XSD** (XML Schema).
2. **message (Paquetes):** Combina los datos anteriores para formar entradas y salidas.
3. **portType (Interfaz):** Es el agrupador clave (como una `interface` Java). Contiene las operaciones.
4. **operation (Funciones):** Define la acción (`sumar`) conectando un mensaje de entrada con uno de salida.

Jerarquía: `types` → componen `message` → usados en `operation` → agrupados en `portType`.

6.3.2 Descripción Concreta (La Red - El "Cómo")

Baja la lógica a tierra definiendo la conexión física.

1. **binding (Protocolo):** Asigna un protocolo concreto (ej. **SOAP** sobre HTTP) al `portType`.
2. **service (Servidor):** Agrupa los puertos.
3. **port (Dirección):** Une un `binding` con una **URL** física (Endpoint).

6.3.3 Resumen Práctico (WSDL vs Java)

Elemento WSDL	Equivalente Java	Función
<code>types</code>	Clases / Beans	Estructura de datos.
<code>message</code>	Argumentos	Paquete de datos de E/S.
<code>portType</code>	<code>interface</code>	Agrupar métodos abstractos.
<code>operation</code>	Método	Firma de la función (<code>sumar</code>).
<code>binding</code>	Implementación	Protocolo (SOAP/HTTP).
<code>service</code>	Instancia	URL del servicio.

6.4 Protocolo SOAP (La Invocación)

SOAP (*Simple Object Access Protocol*) es el estándar XML para invocar servicios. A diferencia de REST, es un protocolo estricto basado en mensajes.

- **Filosofía:** Es **sin estado** (stateless) y, por defecto, **unidireccional** (one-way).
- **Sincronía:** Logra el modelo Petición-Respuesta apoyándose en el transporte (HTTP).

6.4.1 Estructura del Mensaje (El Sobre)

Todo mensaje SOAP es un documento XML con tres partes:

1. **Envelope (Sobre):** Raíz obligatoria que encapsula todo.
2. **Header (Cabecera) - Opcional:** Metadatos (seguridad, transacciones). Puede ser procesado por nodos **intermediarios**.
3. **Body (Cuerpo) - Obligatorio:** La carga útil (payload) con los datos de negocio. Solo para el **destinatario final**.

Atributos Clave de la Cabecera

Permiten controlar el flujo a través de intermediarios:

- `role` (**¿Quién?**): Define qué nodo debe leer la cabecera (`next`, `ultimateReceiver`).
- `mustUnderstand` (**¿Obligatorio?**): Si es `true` y el nodo no entiende la etiqueta, **debe fallar**.

6.4.2 Estilos de Invocación (Body Styles)

SOAP define dos formas de estructurar el XML dentro del Body:

Estilo	Filosofía	Características
Document	"Te envío datos"	Flexible. El XML es arbitrario. No se ve el nombre del método explícitamente.
RPC	"Ejecuta esta función"	Rígido. Simula una llamada a código (<code><metodo><arg1>...</arg1></metodo></code>).

6.4.3 Flujo de Ejecución (Arquitectura)

El viaje del mensaje desde tu código Java hasta el servidor remoto:

1. **Código Cliente:** Llama al método local (`stub.sumar(5,5)`).
2. **Stub Cliente:** Delega en el **Motor SOAP** (JAX-WS).
3. **Motor SOAP:** Serializa a XML (SOAP) y lo pasa al **Cliente HTTP**.
4. **Red:** El mensaje viaja por HTTP (POST).
5. **Router SOAP (Servidor):** Recibe el XML, lo parsea y decide a qué servicio llamar.
6. **Skeleton/Stub Servidor:** Deserializa el XML a objetos Java.
7. **Implementación:** Se ejecuta el código real (`public int sumar...`).

6.5 Ejemplo Práctico: JAX-WS

A. El Servidor (Service Provider)

Se compone de tres partes: Contrato, Lógica y Despliegue.

1. **La Interfaz (Contrato).** Define qué operaciones están disponibles.

```
@WebService
public interface Calculadora {
    @WebMethod
    int sumar(int a, int b);
}
```

2. **La Implementación (Lógica).** El código real que ejecuta la suma.

```
@WebService(endpointInterface = "com.ejemplo.Calculadora")
public class CalculadoraImpl implements Calculadora {
    public int sumar(int a, int b) { return a + b; }
}
```

3. **El Publicador (Despliegue).** Arranca el servidor.

```
public class Publicador {
    public static void main(String[] args) {
        // Publica la implementación en una URL específica
        Endpoint.publish("http://localhost:8080/miCalculadora", new
CalculadoraImpl());
    }
}
```

Método Clave: `Endpoint.publish(url, implementor)`

Es el interruptor de encendido.

- **Arranca:** Inicia un servidor HTTP ligero interno en el puerto 8080.
- **Genera:** Crea el WSDL dinámicamente en memoria.
- **Expone:** Habilita la escucha de peticiones SOAP en esa URL.

B. El Cliente (Service Consumer)

El cliente no tiene la lógica de la suma, solo invoca al servidor remoto.

```
public class Cliente {
    public static void main(String[] args) {
        try {
            // 1. URL del WSDL (El Manual)
            URL wsdlURL = new URL("http://localhost:8080/miCalculadora?wsdl");

            // 2. QName (Identificador Único: Namespace + ServiceName)
            QName qname = new QName("http://ejemplo.com/", "CalculadoraImplService");

            // 3. Crear la Fábrica (Service)
            Service service = Service.create(wsdlURL, qname);

            // 4. Obtener el Proxy (Stub)
            Calculadora proxy = service.getPort(Calculadora.class);

            // 5. Invocación Remota (Marshalling -> Red -> Unmarshalling)
            System.out.println("Resultado: " + proxy.sumar(10, 20));

        } catch (Exception e) { e.printStackTrace(); }
    }
}
```

Métodos Clave del Cliente

Método	Función Técnica
<code>URL (...?wsdl)</code>	Apunta a la ubicación del contrato. Sin el <code>?wsdl</code> al final, no funciona.
<code>QName</code>	Nombre Cualificado. Identifica al servicio exacto dentro del XML del WSDL. Se forma con el <code>TargetNamespace</code> (paquete invertido) y el <code>ServiceName</code> .
<code>Service.create()</code>	La Fábrica. Descarga el WSDL, lo valida y prepara la conexión.
<code>getPort()</code>	El Proxy (Stub). Crea un objeto local falso. Convierte tus llamadas Java (<code>.sumar</code>) en mensajes XML SOAP (Marshalling) y los envía por HTTP.

NOTA

En este tema miraros las cosas por encima, yo le metí mucho detalle y mucha explicación pero porque no entendía un carallo. Pero con entender un poco los conceptos principales malo será.

7. Sistemas de Mensajes

7.1 Introducción y Contexto Histórico

Para comprender la importancia de RabbitMQ, primero debemos entender el problema que resuelve: la **interconectividad en sistemas heterogéneos**.

7.1.1 La era de los Mainframes y la Conexión Punto a Punto

- **Años 60:** La computación estaba dominada por *Mainframes* para operaciones críticas. Las opciones de entrada eran reducidas y la interconectividad entre sistemas era prácticamente inexistente. El procesamiento paralelo tal como lo conocemos hoy no era posible.
- **Años 70:** Se introduce el acceso mediante terminales y el acceso concurrente, permitiendo cierta comunicación entre mainframes a través de redes primitivas.
- **Años 80:** La llegada del PC y las interfaces gráficas incrementó la necesidad de acceder a los datos centralizados en los mainframes.

INFO

Un Mainframe es una computadora de alto rendimiento, gran tamaño y centralizada, diseñada para procesar inmensos volúmenes de datos y transacciones críticas (banca, censos, aerolíneas).

El problema de la escalabilidad ("Spaghetti Integration"): Conectar fuentes y destinos no era sencillo. Cada par de sistemas requería adaptadores específicos para hardware, protocolos y formatos de datos.

- A mayor número de sistemas, mayor número de adaptadores y versiones.
- Esto resultaba en una arquitectura **no escalable** y extremadamente difícil de mantener.

7.1.2 La Solución: Mensajería Empresarial

Como respuesta a este caos de conexiones directas, surge la **mensajería empresarial**.

- **Objetivo:** Transferir información entre sistemas heterogéneos mediante el envío de mensajes, eliminando el acoplamiento directo.
- **Evolución tecnológica:** Se pasó de las Llamadas a Procedimientos Remotos (RPC) (que actúan como middleware tipo COM o CORBA) a los **Sistemas MOM (Message-Oriented Middleware)**.

7.2 Fundamentos de los Sistemas de Mensajes (MOM)

Un sistema de mensajes es un método de comunicación entre componentes de software o aplicaciones que abstrae la complejidad de la red.

Características Clave: La principal ventaja de usar MOM (como RabbitMQ) frente a una conexión directa (como HTTP REST síncrono) reside en el **desacoplamiento**:

- **Desacoplamiento Temporal (Asincronía):** Emisor y receptor no necesitan estar disponibles al mismo tiempo. El emisor envía el mensaje y sigue trabajando; el receptor lo procesará cuando pueda.
- **Desacoplamiento Estructural (Weak Coupling):** Emisor y receptor no necesitan saber nada el uno del otro (ni IP, ni tipo de servidor). Solo necesitan conocer el **formato del mensaje** y el **destino** lógico.
- **Conexión Única:** Solo es necesaria la conexión al MOM, no una conexión mallada entre todos los servicios.

7.3 RabbitMQ: Arquitectura y Protocolos AMQP

RabbitMQ es un middleware de mensajería (Message Broker) concebido en 2007. Aunque soporta múltiples protocolos (STOMP, HTTP), su estándar nativo y más potente es **AMQP 0-9-1**.

7.3.1 ¿Qué es AMQP?

El *Advanced Message Queuing Protocol* es un protocolo asíncrono que garantiza el envío fiable de mensajes. Su función es almacenar mensajes en colas seguras hasta que el receptor se conecte o cumpla los criterios para recibirlo.

7.3.2 Componentes de la Arquitectura AMQP

En un sistema RabbitMQ, la comunicación no es directa de "A" a "B". Fluye a través de componentes específicos:

- **Mensajes:** La unidad de datos. Contiene dos partes:
 - **Contenido (Payload):** los datos en sí (array de bytes)
 - **Clave (Routing Key):** un valor que permite al sistema saber a qué cola debe dirigirse el mensaje.
- **Productor (Producer):** El software que crea y envía el mensaje. Nota: En términos de código, "producir" significa "enviar".
- **Intercambiador (Exchange): Este es el componente crítico.** Es el "cartero" o router. Recibe los mensajes del productor y decide a qué cola enviarlos basándose en reglas (bindings) y en la clave del mensaje.

- **Cola (Queue):** Un buffer o buzón que vive dentro de RabbitMQ. Almacena los mensajes de forma segura hasta su consumo.
- **Consumidor (Consumer):** El software que esperar recibir y procesar los mensajes desde la cola

Importante: El productor **nunca** envía mensajes directamente a la cola. Siempre los envía al Exchange.

7.4 Tipos de Exchange (Estrategias de Enrutamiento)

El comportamiento del sistema depende totalmente del tipo de *Exchange* que configuremos. Tenemos 5 estrategias fundamentales.

7.4.1 Fanout (Difusión)

- **Lógica:** El mensaje se envía a **todas** las colas conectadas al exchange, ignorando la clave del mensaje.
- **Uso:** Ideal para patrones "Publish/Subscribe" donde varios sistemas deben reaccionar al mismo evento.
- **Diagrama conceptual:** Un mensaje entra y se clona hacia la Cola 1 y la Cola 2 simultáneamente.

7.4.2 Direct (Directo)

- **Lógica:** El mensaje va a una cola específica si la **Clave del Mensaje (Routing Key)** coincide **exactamente** con la **Clave de Conexión (Binding Key)** de la cola.
- **Ejemplo:** Si el mensaje tiene clave `abc` y la Cola 1 está atada con `abc`, el mensaje entra. Si la Cola 2 espera `def`, el mensaje `abc` se descarta para esa cola.

7.4.3 Topic (Tema)

- **Lógica:** Similar al Directo, pero permite coincidencias **parciales** usando comodines (wildcards).
- **Ejemplo:** Un mensaje con clave `a.b.c` podría entrar en una cola que escuche `a.*` o `*.b.*`.
- **Diagrama conceptual:** Permite enrutamiento complejo basado en patrones.

7.4.4 Header (Cabecera)

- **Lógica:** Ignora la clave de enrutamiento. En su lugar, inspecciona los **metadatos (headers)** del mensaje para decidir el destino.

7.4.5 Default (Nameless)

- **Lógica:** Es un Exchange predeterminado. Compara la clave del mensaje directamente con el **nombre de la cola**. Si el mensaje tiene clave `cola1`, se entrega a la cola llamada `cola1`.
- **Nota:** Es el comportamiento por defecto si no se especifica exchange.

7.5 Programación con RabbitMQ

7.5.1 Conceptos Fundamentales de Arquitectura

- **Productor (Producer):** Es el programa que *envía* los mensajes.
- **Consumidor (Consumer):** Es el programa que *espera y recibe* los mensajes.
- **Cola (Queue):** Un gran "buffer" o buzón que vive dentro de RabbitMQ donde se almacenan los mensajes.

La tubería: Conexión vs Canal Esta distinción es crítica para entender el código:

- **Conexión (`Connection`):** Es la conexión TCP/IP real entre tu aplicación y el broker RabbitMQ. Es pesada y costosa de establecer (autenticación, handshake).
- **Canal (`Channel`):** Es una conexión virtual dentro de la conexión TCP. La mayoría de las tareas de la API (enviar, recibir, declarar colas) se realizan a través del canal.

Analogía: La *Conexión* es la autopista (cable) que llega al servidor. Los *Canales* son los carriles individuales por donde viajan los coches (mensajes). Se usa una sola conexión y múltiples canales para ser eficientes.

7.5.2 Desarrollo del Productor

El objetivo del productor es conectarse y dejar un mensaje en una cola.

Paso 1: Configurar la Conexión Usamos una `ConnectionFactory` para definir los parámetros del productor.

```
ConnectionFactory factory = new ConnectionFactory();
factory.setHost("localhost"); // Apunta al servidor local
// Si el servidor es remoto, usarías factory.setHost("192.168.1.50");
```

Paso 2: Crear Conexión y Canal (Try-with-resources) Se recomienda usar la estructura `try (...)` de Java para que la conexión y el canal se cierren automáticamente al terminar, liberando recursos.

```
try (Connection connection = factory.newConnection();
    Channel channel = connection.createChannel()) {
```

Paso 3: Declarar la Cola (Idempotencia) Antes de enviar nada, debemos asegurarnos de que el destino existe.

```
channel.queueDeclare(QUEUE_NAME, false, false, false, null);

// queueDeclare(queue, durable, exclusive, autoDelete, arguments)

// 1. queue (String): El nombre de la cola (ej: "mi_cola").
// 2. durable (boolean): false = Si RabbitMQ se reinicia, la cola (y sus mensajes)
DESAPARECEN.
//                               true = La cola se guarda en disco y sobrevive al reinicio.
// 3. exclusive (boolean): false = Otros conexiones/canales pueden acceder a esta
cola.
//                               true = Solo ESTA conexión puede usarla (útil para colas
temporales privadas).
// 4. autoDelete (boolean): false = La cola se queda ahí aunque no haya nadie
escuchando.
//                               true = La cola se borra sola cuando el último consumidor
se desconecta.
// 5. arguments (Map): null = Sin configuración extra (se usa para TTL, longitud
máxima, etc.).
```

Concepto Clave (Idempotencia): Esta operación es *idempotente*. Significa que si la cola ya existe, no hace nada; si no existe, la crea. Esto evita errores si ejecutas el programa múltiples veces.

Paso 4: Publicar el Mensaje Finalmente, enviamos el mensaje convertido a bytes

```
String message = "Hello World!";
channel.basicPublish("", QUEUE_NAME, null, message.getBytes());

//basicPublish(exchange, routingKey, metadatos, body)
```

- El primer parámetro `""` indica que usamos el **Exchange por Defecto**.
- El segundo parámetro usa el nombre de la cola como **Routing Key** (clave de enrutamiento).

7.5.3 Desarrollo del Consumidor (Recv.java)

El consumidor se mantiene a la escucha (listening) para procesar mensajes entrantes.

Paso 1: Configuración Simétrica Al igual que el Productor, el Consumidor debe abrir conexión, canal y declarar la cola.

```
channel.queueDeclare(QueueName, false, false, false, null);
```

¿Por qué repetirlo? Porque en sistemas distribuidos no sabemos quién arranca primero. Si iniciamos el Consumidor antes que el Productor y la cola no existe, el programa fallaría. Declararla en ambos lados garantiza que el "buzón" exista siempre.

Paso 2: Callback de Entrega (Asincronía) El servidor envía mensajes de forma asíncrona (push) al cliente. No usamos un bucle `while` tradicional, sino un **Callback** (una función que se dispara al ocurrir un evento).

Definimos el objeto `DeliverCallback` que almacenará la lógica de "qué hacer cuando llegue un mensaje":

```
DeliverCallback deliverCallback = (consumerTag, delivery) -> {  
    String message = new String(delivery.getBody(), "UTF-8");  
    System.out.println("Recibido: " + message);  
};
```

Paso 3: Iniciar el Consumo Le indicamos al canal que empiece a consumir mensajes de la cola específica usando el callback definido.

```
channel.basicConsume(QueueName, true, deliverCallback, consumerTag -> { });  
  
// basicConsume(queue, autoAck, deliverCallback, cancelCallback)  
  
// 1. queue (String): De qué cola quieres leer.  
// 2. autoAck (boolean): true = "Modo disparo y olvido". RabbitMQ borra el mensaje en  
//    cuanto te lo envía,  
//    sin esperar a que le digas que lo procesaste bien. (Rápido  
//    pero arriesgado).  
//    false = Tienes que enviar un 'ack' manual después de  
//    procesar.  
// 3. deliverCallback: La función (lambda) que se ejecuta CADA VEZ que llega un  
//    mensaje.  
// 4. cancelCallback: La función que se ejecuta si el servidor cancela al consumidor
```

(raro, por ejemplo si la cola se borra).

7.5.4 Estrategias de Enrutamiento (Exchanges)

El verdadero poder de RabbitMQ no es solo guardar mensajes, sino **decidir a quién entregarlos**. Esta decisión la toma el **Exchange** (Intercambiador). El Productor envía al Exchange, y el Exchange reparte a las Colas según el algoritmo elegido.

Default (Sin nombre)

- **Funcionamiento:** Es el comportamiento básico que vimos en el código anterior.
- **Lógica:** Compara la clave del mensaje directamente con el **nombre de la cola**.
- **Código:** `channel.basicPublish("", "nombre_cola", ...)`

Fanout (Difusión)

Concepto: "El Megáfono". El productor grita, y todos los que estén conectados escuchan.. **Caso de uso:** Un marcador deportivo. Si hay gol, actualizas la Web, la App y la TV a la vez.

Productor: no le importa quién escucha. Envía al exchange "deportes".

```
// 1. Declaramos el exchange de tipo FANOUT
channel.exchangeDeclare("exchange_deportes", "fanout");

// 2. Publicamos.
// FÍJATE: RoutingKey está vacía ("" ) porque Fanout la ignora.
String mensaje = "GOL DEL EQUIPO LOCAL";
channel.basicPublish("exchange_deportes", "", null, mensaje.getBytes());
```

Consumidor: aquí cambia la cosa. El consumidor crea una cola temporal y **la conecta** al exchange.

```
channel.exchangeDeclare("exchange_deportes", "fanout");

// Creamos una cola aleatoria, exclusiva y temporal (se borra al desconectar)
String nombreCola = channel.queueDeclare().getQueue();

// BINDING: "Conecta mi cola temporal al exchange de deportes"
channel.queueBind(nombreCola, "exchange_deportes", "");

// Consumimos
channel.basicConsume(nombreCola, true, deliverCallback, consumerTag -> {});
```

Direct (Directo)

Concepto: "El Cartero Exacto". El mensaje lleva una etiqueta y solo entra en la cola que espera esa etiqueta exacta. **Caso de uso:** Sistema de Logs. Un consumidor guarda en disco solo los "errores", otro imprime en pantalla "info" y "errores".

Productor: Pone una etiqueta específica (`routingKey`) al mensaje.

```
// 1. Tipo DIRECT
channel.exchangeDeclare("exchange_logs", "direct");

// 2. Enviamos un error grave
String mensaje = "Fallo crítico en base de datos";
String severidad = "error"; // Esta es la Routing Key
channel.basicPublish("exchange_logs", severidad, null, mensaje.getBytes());
```

Consumidor: solo quiere recibir lo que venga con la etiqueta "error"

```
channel.exchangeDeclare("exchange_logs", "direct");
String nombreCola = channel.queueDeclare().getQueue();

// BINDING: "Solo envíame mensajes si la routing key es EXACTAMENTE 'error'"
channel.queueBind(nombreCola, "exchange_logs", "error");

channel.basicConsume(nombreCola, true, deliverCallback, consumerTag -> {});
```

Topic (Tema)

Concepto: "El Patrón Inteligente". Permite filtrar por partes usando comodines (`*` para una palabra, `#` para varias). **Caso de uso:** Noticias. Categorías como `deportes.futbol.madrid` o `politica.internacional.usa`.

Productor: Usa claves jerárquicas separadas por puntos.

```
channel.exchangeDeclare("exchange_noticias", "topic");

String mensaje = "El Madrid gana la liga";
// Clave: Categoría.Subcategoría.Equipo
channel.basicPublish("exchange_noticias", "deportes.futbol.madrid", null,
mensaje.getBytes());
```

Consumidor A (Fanático del fútbol en general): Quiere todo lo que sea fútbol, no importa el equipo.

```
// BINDING: El asterisco (*) sustituye a UNA palabra (el equipo)
// Recibirá: deportes.futbol.madrid, deportes.futbol.barca
channel.queueBind(nombreCola, "exchange_noticias", "deportes.futbol.*");
```

Consumidor B (Periodista de Deportes): Quiere absolutamente todo lo que empiece por "deportes".

```
// BINDING: La almohadilla (#) sustituye a CUALQUIER cantidad de palabras
// Recibirá: deportes.futbol.madrid, deportes.tenis, deportes.waterpolo.femenino
channel.queueBind(nombreCola, "exchange_noticias", "deportes.#");
```

Header (Cabecera)

Concepto: "Enrutamiento por Metadatos". Ignora la routing key y mira las propiedades del mensaje. Es menos común en cursos introductorios pero muy potente. **Caso de uso:** Procesar archivos. Si el mensaje es XML va a un lado, si es JSON a otro.

Productor: Configura un mapa de propiedades.

```
channel.exchangeDeclare("exchange_archivos", "headers");

Map<String, Object> cabeceras = new HashMap<>();
cabeceras.put("formato", "pdf");
cabeceras.put("tipo", "informe");

AMQP.BasicProperties props = new AMQP.BasicProperties.Builder()
    .headers(cabeceras)
    .build();

channel.basicPublish("exchange_archivos", "", props, mensaje.getBytes());
```

Consumidor: Dice "Dame mensajes que tengan estas cabeceras".

```
Map<String, Object> bindingArgs = new HashMap<>();
bindingArgs.put("x-match", "all"); // Deben coincidir TODAS las reglas
bindingArgs.put("formato", "pdf");

channel.queueBind(nombreCola, "exchange_archivos", "", bindingArgs);
```

8. Agentes

NOTA

La cantidad de relleno que hay en este tema es histórico y mira que intenté quitar la mayor cantidad de mierda posible.

8.1 ¿Qué es un Agente? (Conceptos Básicos)

Imagina un **mayordomo digital**. A diferencia de un programa normal (como una calculadora) que espera a que tú le des órdenes paso a paso, el mayordomo sabe qué quieres y actúa por su cuenta para conseguirlo.

Definición Clave: Un agente es un sistema informático situado en un entorno, capaz de realizar acciones de forma **autónoma** y flexible para cumplir los objetivos de su "dueño" (usuario).

Diferencia principal con el software tradicional

- **Objeto/Programa normal:** "¿Qué hago ahora?" (Es pasivo, espera órdenes).
- **Agente:** "¿Qué necesito hacer para conseguir mi meta?" (Es activo, toma decisiones).

8.2 Características Fundamentales

Para que un programa sea considerado un "Agente", debe cumplir estas propiedades (especialmente las tres primeras):

1. **Autonomía (La más importante):** Opera sin intervención directa. Tiene control sobre su estado interno y sus acciones.
2. **Reactividad:** Percibe su entorno y responde a cambios rápidamente (ej: si un robot ve una pared, frena).
3. **Proactividad (Iniciativa):** No solo reacciona; toma la iniciativa para cumplir objetivos a largo plazo (ej: un agente de bolsa que compra acciones porque predice una subida, no solo porque el precio cambió hace un segundo).
4. **Habilidad Social:** Interactúa con otros agentes o humanos (coopera, negocia o compite).
5. **Movilidad:** El agente tiene la capacidad de viajar de nodo en nodo de la red preservando su estado

8.3 El Entorno

El entorno es "el mundo" donde vive el agente.

Clasificar el entorno es vital porque **determina la dificultad** de diseñar el agente. Cuanto más difícil es el entorno, más inteligente debe ser el agente.

Propiedad del Entorno	Definición Sencilla	Ejemplo Fácil (Ajedrez)	Ejemplo Difícil (Conducir un Taxi)
Accesibilidad	¿El agente puede ver TODO el escenario?	Totalmente Accesible: Ves todo el tablero.	Inaccesible: No ves qué hay tras la esquina o qué piensan otros conductores.
Determinismo	Si hago una acción, ¿el resultado es 100% seguro?	Determinista: Si muevo el peón, se queda ahí. No hay azar.	No Determinista (Estocástico): Si giro el volante, puede que el coche patine si hay aceite. Hay incertidumbre.
Episodicidad	¿Lo que hice antes afecta a lo que hago ahora?	Secuencial (No episódico): Un mal movimiento al inicio afecta el final del juego.	Episódico: Identificar fotos defectuosas en una cadena de montaje (la foto anterior no afecta a la actual).
Dinamismo	¿El mundo cambia mientras el agente "piensa"?	Estático: El tablero no se mueve hasta que tú mueves.	Dinámico: Mientras decides si frenar, el coche de delante sigue avanzando.
Continuidad	¿Las opciones son finitas o infinitas?	Discreto: Hay un número fijo de casillas y movimientos.	Continuo: La velocidad y el ángulo del volante tienen infinitos valores posibles.

Resumen: El entorno más difícil es **Inaccesible, No determinista, Dinámico y Continuo** (como el mundo real). El más fácil es **Accesible, Determinista, Estático y Discreto** (como un puzzle).

8.4 Arquitecturas de Agentes

La arquitectura es el "cerebro" del agente. Define cómo pasa de **Percibir (sensores)** a **Actuar (actuadores)**. Las ordenamos de la más "tonta" a la más compleja.

A. Arquitecturas Reactivas (Estímulo-Respuesta)

Son como insectos. No "piensan" ni tienen memoria compleja.

- **Funcionamiento:** Si ocurre X, haz Y.
- **Ventaja:** Son rapidísimos.

- **Desventaja:** No pueden planificar a futuro. Si el entorno cambia de forma que sus reglas no cubren, fallan.
- **Ejemplo:** Un termostato ($\text{Si } T < 20^\circ \rightarrow \text{Encender calefacción}$).
- **Subsunción (Rodney Brooks):** Es un tipo famoso de arquitectura reactiva organizada en capas de comportamientos simples (ej: evitar obstáculos) que, al sumarse, crean un comportamiento complejo (caminar).

B. Arquitecturas Basadas en Lógica (El enfoque purista)

Es el modelo teórico clásico. El agente funciona como un **matemático**: todo lo que percibe lo convierte en fórmulas para "demostrar" qué acción debe tomar.

- **Funcionamiento:** Su "cerebro" es una base de datos de conocimiento lógico.
 1. **Representación:** Usa **Lógica de Primer Orden** para describir el estado del mundo.
 2. **Deducción:** Usa reglas lógicas para deducir qué acción realizar.
- El Ciclo Lógico (Formal): El agente usa tres funciones matemáticas clave:
 1. $Ver(s)$: Transforma lo que ven los sensores (s) en una percepción (P).
 2. $Próximo(D, P)$: Actualiza su base de datos interna (D) combinando lo que ya sabía con la nueva percepción.
 3. $Acción(D)$: Consulta su base de datos y decide lógicamente qué acción (A) ejecutar.
- **Ventajas:**
 - **Elegancia:** Es una representación muy clara, limpia y formal.
- **Desventajas:**
 - **Lentitud extrema:** La deducción lógica tiene una complejidad temporal muy alta.
 - **Abstracción difícil:** Es muy complicado traducir el mundo real (que es caótico) a símbolos lógicos perfectos.

C. Arquitecturas Deliberativas

Son como filósofos o jugadores de ajedrez.

- **Funcionamiento:** Tienen un modelo simbólico del mundo (un mapa mental). Antes de actuar, razonan lógicamente para buscar un plan que les lleve a su meta.
- **Ventaja:** Pueden resolver problemas complejos y explicar por qué hicieron algo.
- **Desventaja:** Son lentos. En entornos de tiempo real (dinámicos), pueden quedarse "pensando" mientras el mundo cambia y los atropella.

D. Arquitectura B.D.I. (Belief, Desire, Intention)

Es la arquitectura deliberativa más famosa porque imita el razonamiento humano práctico.

1. **Creencias (Beliefs):** Lo que el agente *sabe* o cree saber del mundo (Información).
 2. **Deseos (Desires):** Lo que el agente *quiere* conseguir (Metas posibles).
 3. **Intenciones (Intentions):** Los deseos por los que ha decidido *actuar* ahora mismo (Plan actual).
- **El dilema del BDI:**
 - *Agente Audaz:* Elige un plan y lo sigue ciegamente hasta terminar o fallar. (Rápido, pero terco).
 - *Agente Cauto:* Para a cada paso para ver si el plan sigue siendo válido. (Seguro, pero muy lento).

E. Arquitecturas Híbridas (Lo mejor de ambos mundos)

La mayoría de sistemas modernos usan esto. Combinan una parte reactiva (para reflejos rápidos) y una deliberativa (para planes a largo plazo).

Se organizan en Capas:

1. **Capas Horizontales:** Todas las capas tienen acceso a los sensores y actuadores.
 - *Problema:* Necesitas un "árbitro" central que decida a qué capa hacer caso si se contradicen (Cuello de botella).
2. **Capas Verticales:** La información pasa como una cadena de mando.
 - *Problema:* Si una capa falla, se rompe toda la cadena.

8.5 Sistemas Multiagente (Sociedades)

Cuando juntamos varios agentes, pasamos de diseñar individuos a diseñar sociedades. Aquí surgen conceptos clave:

- **Heterogeneidad:** Agentes distintos trabajando juntos (ej: un agente busca vuelos, otro reserva hoteles).
- **Cooperación vs. Competición:** ¿Trabajan juntos para un fin común (equipo de fútbol) o compiten por recursos limitados (subasta)?
- **Comunicación:** Necesitan un lenguaje común para negociar o delegar tareas.

La principal diferencia entre los sistemas multiagente y las redes P2P es la **homogeneidad**, el nivel de autonomía e inteligencia. EN una red P2P los nodos que intervienen son todos homogéneos, su comportamiento es **rígido y reactivo**, todos los nodos realizan las mismas funciones y **colaboran entre sí**. Por el otro lado, los agentes son **heterogéneos**, cada uno se especializa en realizar una tarea o función y puede **competir o colaborar** con otros agentes. Además su comportamiento es **proactivo y autónomo**, no solo reactivo.

¿Por qué usar Agentes? (Utilidad)

Son ideales para sistemas donde **no hay un control centralizado** o el problema es muy grande y cambiante.

- **Escalabilidad:** Puedes añadir más agentes si hay más trabajo.
- **Robustez:** Si un agente falla, los otros pueden seguir trabajando.
- **Abstracción:** Es más natural modelar un mercado con "agentes compradores y vendedores" que con un código monolítico gigante.

8.6 Comunicación entre Agentes

A diferencia de los objetos (que invocan métodos), los agentes se comunican para **negociar, cooperar y delegar**. No se trata solo de mover datos, sino de provocar una reacción o un cambio en el otro.

8.6.1 Teoría de los Actos del Habla (La Base Teórica)

La comunicación de agentes se basa en la idea de que **"decir algo es hacer algo"**. El mensaje no es solo información, lleva una intención.

Se divide en tres dimensiones:

1. **Locución:** El acto físico de emitir el mensaje (decir la frase).
2. **Ilocución (La más importante):** La **intención** real del mensaje. ¿Es una orden? ¿Una promesa? ¿Una pregunta? Esto define el "Performative".
3. **Perlocución:** El efecto que se logra en el receptor (convencer, asustar, informar).

Clasificación de la Intención (Ilocución)

- **Asertivas:** Informar sobre hechos ("El cielo es azul").
- **Directivas:** Intentar que el receptor haga algo ("Cierra la puerta", "¿Qué hora es?").
- **Comisivas:** Comprometerse a hacer algo ("Prometo ir mañana").
- **Permisivas/Prohibitivas:** Dar o quitar autorizaciones.
- **Declarativas:** Cambian la realidad con solo decirlo ("Os declaro marido y mujer").
- **Expresivas:** Muestran estado emocional ("Estoy cansado").

Objetivo del diseño: Se busca **completitud** (poder decir todo lo necesario), **simplicidad** (que no sea imposible de programar) y **concisión** (evitar ambigüedad).

8.6.2 La "Pila" de Comunicación

Para que dos agentes se entiendan, deben coincidir en varios niveles, de abajo a arriba:

Nivel 1: Transporte y Conexión (¿Cómo llega el mensaje?)

Es la tubería física. Los agentes no se preocupan de los bytes, delegan en un **Servicio de Transporte**.

- **Mecanismos:** Puede ser síncrono (espero respuesta) o asíncrono (envío y sigo trabajando). Puede ser a uno (Unicast), a varios (Multicast) o a todos (Broadcast).
- **Tecnologías:** CORBA, RMI, DCOM.
- **Funciones:** Garantizar que el mensaje llega, detectar errores y manejar las direcciones (nombres) de los agentes.

Nivel 2: Lenguaje de Comunicación (ACL)

Es la sintaxis. Define la estructura del mensaje para que sea comprensible.

- **Estándares:** **FIPA-ACL** (el más usado) o KQML.
- **Requisito clave:** Debe ser formal y tener una semántica bien definida.

INFO

Un **estándar** es simplemente un **conjunto de reglas acordadas** que todo el mundo decide seguir para que cosas diferentes puedan funcionar juntas.

Por ejemplo los enchufes de las paredes, en Europa todos acordaron usar dos clavijas redondas. Eso es un estándar. Si cada fabricante de neveras hiciese un enchufe diferente sería un puto caos. En agentes necesitas estándares para que un agente programado en Java por una empresa en España pueda hablar con un agente programado en Python por una empresa en Japón.

INFO

FIPA (Foundation for Intelligent Physical Agents) es la organización que **escribe** esos estándares. Es como la **FIFA** en fútbol, no juega los partidos, pero escribe el reglamento. La **FIPA** dice: "así es como debe hablar una gente para que el resto del mundo lo entienda".

INFO

ACL (Agent Communication Language) es el lenguaje estándar que inventó FIPA. Pero no se refiere al idioma, sino a la **estructura del mensaje**. Piensa en un **formulario oficial** o en el **sobre de una carta**. ACL define los campos obligatorios para que el mensaje sea válido, independientemente de lo que quieras decir.

Nivel 3: Ontologías (El Vocabulario)

Los agentes no tienen "sentido común". Si el Agente A le pide al Agente B: "*Tráeme un gato*", hay un problema grave de ambigüedad.

- ¿Se refiere a un **animal** (mascota)?
- ¿Se refiere a una **herramienta** (para levantar el coche)?

La Ontología es la solución a este problema. Es una especificación formal y explícita de un vocabulario compartido. Define **QUÉ** significan las palabras en un contexto concreto y cómo se relacionan entre sí.

Para que dos agentes se entiendan, ambos deben acordar usar la misma ontología (el mismo diccionario) antes de hablar.

Ejemplo práctico:

1. Sin Ontología:

- Agente A: "Envía planta".
- Agente B: (Confuso) "¿Una flor o una fábrica?"

2. Con Ontología de Botánica:

- La ontología define: **Planta** es un **Ser Vivo** que realiza fotosíntesis.
- Agente A: "Envía planta (Ontología: Botánica)"
- Agente B: "Entendido, enviando un ficus".

3. Con Ontología Industrial:

- La ontología define: **Planta** es un **Edificio** donde se manufacturan bienes.
- Agente A: "Envía planta (Ontología: Industrial)".
- Agente B: "Entendido, enviando planos de la fábrica de Ford".

Resumen: La ontología da **semántica** (significado real) a los datos. Sin ella, los agentes solo se pasan palabras vacías.

Nivel 4: Protocolos de Interacción (La Conversación)

Son patrones predefinidos de mensajes. No basta con enviar una frase, hay que saber qué se espera después. (Ej: Si pregunto, espero una respuesta).

8.6.3 El Estándar FIPA-ACL

Es el estándar de facto para la comunicación de agentes. Un mensaje FIPA tiene una estructura "tipo sobre de carta" que incluye:

- **Performative:** El tipo de acto (REQUEST, INFORM, AGREE).
- **Sender / Receiver:** Quién envía y quién recibe.
- **Content:** El contenido real de la comunicación.
- **Language & Ontology:** Qué idioma y diccionario se usan para entender el contenido.
- **Protocol:** A qué conversación pertenece esto.

Requerimientos para ser un Agente FIPA

1. **Manejo de errores:** Si recibes un mensaje que no entiendes, **debes** responder con `not-understood`. Nunca ignorarlo silenciosamente.
2. **Fidelidad:** Si usas un acto comunicativo estándar (ej: `INFORM`), debes cumplir estrictamente su definición oficial.
3. **Flexibilidad:** No estás obligado a implementar todos los protocolos, solo el subconjunto que necesites.

8.6.4 Protocolos FIPA (Patrones de Conversación)

Los protocolos definen el "baile" de mensajes entre agentes. Un protocolo es el **patrón de comportamiento** esperado en una conversación. No basta con enviar un mensaje, hay que saber qué respuesta esperar y en qué orden. Los más importantes son:

A. Protocolos Básicos (Petición e Información)

- **FIPA-Query:** Para preguntar. (`query-if` para sí/no, `query-ref` para obtener un dato concreto).
- **FIPA-Request:** Ordenar a otro que haga algo. El otro puede aceptar (`agree`) o rechazar (`refuse`).
- **FIPA-Request-When:** Como el anterior, pero la acción se hace solo cuando se cumple una condición futura.
- **FIPA-Subscribe:** "Avísame cada vez que pase X".

B. Protocolos de Mercado (Negociación)

- **FIPA-Contract-Net (Red de Contratos):**
 1. *Manager* envía una oferta de trabajo (`cfp`: call for proposal).
 2. *Contratistas* envían sus presupuestos (`propose`) o rechazan (`refuse`).
 3. *Manager* elige la mejor y la acepta (`accept-proposal`) rechazando el resto.
 - *Versión Iterated:* Se hacen varias rondas para afinar el precio.
- **FIPA-English-Auction:** Subasta clásica (hacia arriba). El subastador sube el precio hasta que nadie más puje.
- **FIPA-Dutch-Auction:** Subasta holandesa (hacia abajo). Empieza muy alto y baja hasta que alguien dice "mío".

INFORM: Es el acto más básico.

- **Qué significa:** "Te cuento esto que sé y asumo que es verdad".
- **Ejemplo:** Un sensor de temperatura dice: "*Hace 25 grados*".
- **Ojo:** El receptor no está obligado a hacer nada, solo "toma nota".

CFP (Call For Proposal): El "Llamamiento".

- **Qué significa:** "Tengo un problema, ¿quién me lo soluciona y qué condiciones me ponéis?". No es una orden, es una invitación a negociar.
- **Ejemplo:** "Necesito un taxi para ir al aeropuerto, ¿quién me lleva y por cuánto?"

PROPOSE: La "Oferta".

- **Qué significa:** Respuesta a un CFP. El agente dice que puede hacerlo y bajo qué condiciones.
- **Ejemplo:** "Yo te llevo por 20 euros".

ACCEPT-PROPOSAL: El "Trato hecho".

- **Qué significa:** El que envió el CFP ha recibido varias ofertas (proposals) y elige esta.
- **Ejemplo:** "Vale, tú me llevas por 20 euros. Vente".

REJECT-PROPOSAL: El "No, gracias".

- **Qué significa:** Has recibido la oferta, pero no te interesa (porque es cara, porque ya elegiste a otro, etc.).
- **Ejemplo:** "30 euros es muy caro, paso de ti".

DANGER

Se dice que el profesor tiene este protocolo tatuado en la espalda y que incluso ha llamado a sus hijos FIPA e English Auction. Si no lo usas en sus prácticas se calienta una burrada, a pesar de que haya sido creado por un medico holandés en el 1836 y sea la mierda más ineficiente nunca jamás creada.

C. Protocolos de Intermediación

Cuando no sabes quién puede hacer el trabajo, usas un intermediario.

- **FIPA-Brokering:** Le pides algo al bróker, él busca al agente adecuado, consigue la respuesta y **te la devuelve él mismo**. (Tú no ves al agente final).
- **FIPA-Recruiting:** Le pides al reclutador que busque a alguien. El reclutador le pasa tu petición al agente final y este **te responde a ti directamente**. (El reclutador solo os presenta).

8.7 JADE (Java Agent Development Framework)

8.7.1 ¿Qué es y para que sirve?

JADE es un **framework** de código abierto que facilita la creación de sistemas distribuidos multiagente.

INFO

Es un **esqueleto de software** o estructura base que proporciona el código necesario para las tareas comunes y repetitivas (como la comunicación, seguridad o gestión de errores). A diferencia de una simple librería (donde tú llamas al código), el framework **te impone unas reglas** y una forma de trabajar, encargándose él de "llamar" a tu código cuando es necesario. Su objetivo es que el programador no empiece desde cero y se centre solo en la lógica específica de su problema. JADE te da la "casa" (el sistema de mensajería, el ciclo de vida, la conexión entre ordenadores). Tú solo pones los "muebles" (la inteligencia del agente y sus decisiones).

Piensa en JADE como el **Sistema Operativo** de los agentes. En lugar de tener que programar desde cero cómo se envían los bytes por la red o cómo se encuentra dos programas, JADE te da esa infraestructura hecha para que tú solo te preocupes de la inteligencia del agente.

- Está escrito en **Java**
- Cumple totalmente con **FIPA**. Lo que garantiza que un agente JADE puede hablar con agentes de otras plataformas si también son FIPA
- **Servicios que regala:** Ciclo de vida (nacer/morir), movilidad (viajar entre ordenadores), páginas amarillas (búsqueda), mensajería y seguridad.

8.7.2 La Arquitectura JADE (El "Mundo")

Imagina una **Universidad (La Plataforma)**. Una universidad no es un solo edificio. Es una institución formada por varios edificios repartidos por la ciudad (Campus Norte, Campus Sur, Rectorado).

- **La Plataforma (La Universidad):** Es el concepto abstracto. Es el **grupo total** de ordenadores que están conectados trabajando juntos en tu sistema de agentes.
- **El Contenedor (Cada Edificio):** Es lo que ejecutas en cada ordenador físico.
 - Si tienes 3 ordenadores conectados formando tu sistema JADE, tienes **1 Plataforma** distribuida en **3 Contenedores**.
 - Dentro de cada "Edificio" (Contenedor) viven las personas (Agentes).
- **Main Container:** es el primer contenedor que se arranca. Es el cerebro de la plataforma. Si este se cae, la plataforma muere. Contiene dos agentes especiales obligatorios:
 - **AMS (Agent Management System):** la policía. Controla quién entra, quién sale, crea agentes y mata agentes.
 - **DF (Directory Facilitator):** las páginas amarillas. Los agentes se anuncian aquí (Vendo servicios de impresión) para que los encuentren.

Comando para lanzar JADE: `java jade.Boot -gui` (Arranca el Main Container y la interfaz gráfica).

INFO

Java inventó una "máquina virtual". Es un programa que instalas en tu ordenador y actúa como un **traductor en tiempo real**.

- Tú escribes código Java (JADE).
- La JVM lee ese código y se lo traduce a TU ordenador específico.

Cuando decimos "Un contenedor JADE corre sobre una JVM", significa que para arrancar JADE, necesitas tener instalado Java y ejecutar el proceso. **1 Contenedor = 1 Proceso de Java abierto en tu ordenador.**

Además tenemos otros agentes disponibles:

- **RMA (Remote Management Agent):** Es la consola principal (la ventana que se abre al inicio). Controla la plataforma, mata agentes, lanza otros agentes y monitoriza contenedores remotos.
- **DF (Directory Facilitator):** Son las "**Páginas Amarillas**" del sistema.
 - **Función:** Permite que los agentes se encuentren entre sí por lo que *hacen* y no por su nombre.
 - **Ejemplo:** Un agente se registra aquí diciendo "Yo ofrezco el servicio de impresión". Otro agente que necesite imprimir consultará al DF: "¿Quién imprime?", y el DF le dará la dirección del primero.
- **Sniffer Agent:** Fundamental para entender qué pasa. Muestra un diagrama visual de quién envía mensajes a quién (como flechas entre líneas de vida en un diagrama de secuencia).
- **Introspector Agent:** Es como una radiografía. Te permite ver el **estado interno** de un agente: qué comportamientos (`Behaviours`) está ejecutando ahora mismo, la cola de mensajes pendientes y el valor de sus variables.
- **Log Manager Agent:** Gestiona los logs de la plataforma en tiempo de ejecución. Permite cambiar qué nivel de detalle se guarda sin tener que reiniciar el programa.
- **Dummy Agent:** Un agente manual para enviar mensajes de prueba y comprobar si tu sistema responde correctamente.

8.7.3 Programación de un Agente

Un agente es un componente activo con un nombre único y un **AID** (Agent ID)

El ciclo de vida de un agente es el siguiente:

- **Nacimiento (`setup`):** se ejecuta una sola vez al arrancar. Aquí inicializas variables y, lo más importante, **añades los primeros comportamientos** (`addBehaviour`) a la "piscina" de tareas.
- **El Bucle Principal (El corazón del Agente):** el agente entra en un bucle `while` que dura toda su vida. ¿El agente ha muerto (`doDelete` llamado)? Si ha muerto sale del bucle y va a `takeDown()`, si no continúa.

- **Ejecución de Comportamientos:** el agente coge el siguiente comportamiento activo de su lista, ejecuta el método `b.action()` de ese comportamiento (solo se ejecuta un paso, no todo el proceso si es largo).
- **Verificación:** Pregunta `b.done()`. Si devuelve **True** el comportamiento ha terminado y se elimina de la lista, si devuelve **False** el comportamiento sigue vivo y se queda en la lista para la siguiente vuelta
- **Muerte (`takeDown()`):** se ejecuta antes de desaparecer. Aquí cierra conexiones a bases de datos, te desregistras de las páginas amarillas, etc.

Para crear un agente, heredas de la clase `jade.core.Agent`

```
import jade.core.Agent;

public class MiAgente extends Agent {

    // 1. SETUP: Equivale al "main". Se ejecuta al nacer.
    protected void setup() {
        System.out.println("Hola, soy el agente " + getAID().getName());
        // Aquí se añaden los comportamientos (el cerebro)
        addBehaviour(new MiComportamiento());
    }

    // 2. TAKEDOWN: Se ejecuta justo antes de morir (limpieza).
    protected void takeDown() {
        System.out.println("Adios mundo cruel.");
    }
}
```

8.7.4 Los comportamientos

Un agente JADE funciona en un SOLO HILO de Java. ¿Cómo puede hacer varias cosas a la vez (escuchar mensajes y calcular datos)? Usando **comportamientos colaborativos, no apropiativos**. Para realizar esto, el programador debe definir cuando el comportamiento debe **ceder CPU** de forma voluntaria a otros comportamientos (normalmente cuando se usan operaciones bloqueantes, esto se explica más abajo).

Todo trabajo que hace el agente debe ir dentro de una clase que herede de `jade.core.behaviours.Behaviour`. Tiene dos métodos clave:

- `action()`: lo que hace el agente
- `done()`: devuelve `true` si ha terminado para siempre, o `false` si debe volver a ejecutarse

Tipos de Comportamientos

Tipo	Clase Java	Descripción	done() devuelve...	Ejemplo
OneShot	OneShotBehaviour	Se ejecuta una sola vez y termina.	true	Enviar un email de bienvenida.
Cíclico	CyclicBehaviour	Nunca termina. Se repite infinitamente.	false	Escuchar mensajes entrantes.
Waker	WakerBehaviour	Se ejecuta una vez tras un tiempo de espera (Temporizador).	true	"Despiértame en 10 segundos".
Ticker	TickerBehaviour	Se ejecuta repetidamente cada X tiempo (Metrónomo).	false	Comprobar el precio de una acción cada minuto.
Complejo	FSMBehaviour, etc.	Máquinas de estados. Cambia de tarea según lo que pase.	Depende	Lógica compleja de negocio.

8.7.5 Comunicación en JADE

Los agentes se comunican enviando objetos `ACLMessage` de forma asíncrona.

Enviar un mensaje

```
ACLMessage msg = new ACLMessage(ACLMessage.INFORM); // 1. Performative
msg.addReceiver(new AID("AgenteB", AID.ISLOCALNAME)); // 2. Receptor
msg.setLanguage("Castellano");
msg.setContent("Está lloviendo"); // 3. Contenido
send(msg); // 4. Enviar
```

Recibir mensajes

El agente tiene una cola de mensajes

Forma no bloqueante:

```
ACLMessage msg = receive(); // Mira si hay mensaje
if (msg != null) {
    // Procesar mensaje
} else {
    block(); // ¡IMPORTANTE!
}
```

- `block()`: Saca el comportamiento de la lista de ejecución y pone el agente a dormir para ahorrar CPU. El agente se despierta solo cuando llega un mensaje nuevo.
- `MessageTemplate`: Sirve para filtrar. "Solo quiero leer mensajes que vengan de Juan":
`receive(MessageTemplate.MatchSender(juan))`.

Forma bloqueante:

- Detiene toda la ejecución hasta que llega un mensaje.
- **Peligro:** Como congela el único hilo del agente, **nunca lo uses dentro de un Behaviour**. Úsalo solo en `setup()` o en hilos separados.

JADE: DIFERENCIA ENTRE `block()` Y `blockingReceive()`

1. `myBehaviour.block()` (Enfoque Colaborativo)

- **Mecanismo:** Es un bloqueo **lógico**. Marca el comportamiento como "en espera" y devuelve el control al planificador del agente.
- **El Hilo (Thread): No se detiene.** El agente aprovecha para ejecutar otros comportamientos concurrentes.
- **Despertador:** El comportamiento se reactiva automáticamente cuando llega un **nuevo mensaje** a la cola del agente.
- **Uso:** Es la forma correcta de esperar mensajes dentro del método `action()`.

2. `myAgent.blockingReceive()` (Enfoque Bloqueante)

- **Mecanismo:** Es una llamada **síncrona**.
- **El Hilo (Thread): Se bloquea físicamente.** Detiene por completo el flujo de ejecución del hilo Java del agente.
- **Consecuencia:** El agente se "congela" totalmente. Ningún otro comportamiento puede ejecutarse hasta que llegue el mensaje esperado.
- **Uso:** Evitar en comportamientos cíclicos. Útil solo en la inicialización (`setup()`) o fases lineales estrictas.

9. Ontologías

9.1 El Problema de la Comunicación

Para que dos agentes colaboren, no basta con que se envíen mensajes; deben **entenderse**. Tenemos 3 estrategias evolutivas para enviar datos dentro de un mensaje ACL.

9.1.1 Cadenas de Texto (Strings)

- **Método:** Escribir todo en un String plano. Ejemplo: `"valor=10; fecha=20/10"`.
- **Problema:** Es fácil para humanos, pero ineficiente para máquinas. Requiere *parsear* (analizar sintácticamente) el texto, lo cual es propenso a errores y computacionalmente costoso.

9.1.2 Objetos Serializables (Java Serialization)

- **Método:** Enviar objetos Java directamente (`implements Serializable`).
- **Problema:** Crea un **acoplamiento fuerte**. Todos los agentes deben estar programados en Java. Si un agente está en Python o C++, no podrá "deserializar" el objeto Java.

9.1.3 Ontologías (La Solución Estándar)

- **Método:** Definir un modelo formal del dominio (clases y jerarquías) independiente del lenguaje de programación.
- **Ventaja:** Proporciona **interoperabilidad real**. Un agente en Java y otro en C++ pueden entender el mismo concepto "Oferta" si comparten la misma ontología, independientemente de cómo lo representen internamente.

9.2 Definición y Teoría de Ontologías

Una ontología no es solo código; es un contrato semántico. Una ontología es una especificación explícita de una conceptualización. Define el vocabulario (tipos, propiedades y relaciones) de un dominio específico sin ambigüedad.

Qué ganamos usándolas?

1. **Eliminación de Ambigüedad:** Impide construir mensajes "mal formados". Si la ontología dice que una `Persona` tiene `Nombre` y `Edad`, no puedes enviar una `Persona` con `Marca` y `Modelo`.
2. **Validación Semántica:** Si un agente recibe un mensaje que viola la ontología, lo descarta automáticamente (en JADE lanza una excepción), protegiendo la lógica del sistema.

3. **Desacoplamiento:** La ontología define el *qué* (estructura), separándolo del *cómo* (implementación en Java/C#).

9.3 Arquitectura de Ontologías en JADE

JADE implementa este estándar a través del paquete `jade.content`. Para entenderlo, debemos diferenciar tres tipos de elementos que componen cualquier ontología en este framework:

- **Concept (Concepto):**
 - Representa objetos o entidades pasivas del dominio
 - Ejemplo: `Persona`, `Vinilo`, `Coche`
 - *Nota técnica:* Suelen implementarse como Java Beans (getters/setters) y pueden contener otros conceptos.
- **Predicate (Predicado):**
 - Representa hechos o estados del mundo que pueden ser **Verdaderos o Falsos**
 - Ejemplo: `Trabajar`, `EsDueñoDe`
 - Relaciona conceptos entre sí
- **AgentAction (Acción del Agente):**
 - Es un tipo especial de Concepto que representa una orden o acción a realizar (Verbos imperativos)
 - Ejemplo: `Ofertar`, `Pedir`, `Vender`
 - *Importante:* A diferencia de los conceptos normales, estos sí tienen sentido como el "contenido principal" de un mensaje comunicativo.

Detalle de Implementación sobre Acciones: Un `AgentAction` nunca viaja solo. En JADE, debe envolverse en una clase genérica llamada `Action` que incluye quién realiza la acción (`AID` del agente) y la acción en sí.

9.4 Estructura Interna: Esquema vs Beans

Para que JADE maneje las ontologías, divide el trabajo en dos capas. Es vital no confundirlas:

9.4.1 La Clase `Ontology` (El Mapa)

- Hereda de `jade.content.onto.Ontology`.
- Define la **estructura abstracta** (los esquemas). Dice: "En este mundo existe un concepto 'Persona' que tiene un slot 'nombre'".
- Es un **Singleton**: Solo debe haber una instancia de esta clase en toda la ejecución, ya que el esquema no cambia.

9.4.2 Los Beans (Los Datos)

- Son clases Java normales que implementan interfaces (`Concept`, `Predicate`, `AgentAction`).
- Contienen los **datos reales**: "Esta instancia específica es la Persona 'Juan'".
- Se crean y destruyen continuamente durante la ejecución .

9.5 El Proceso de Comunicación: El ContentManager

¿Cómo viaja un objeto Java por la red? Aquí entra el **Lenguaje de Contenido (Content Language)**.

9.5.1 Lenguajes Soportados

JADE traduce los objetos a una secuencia de bytes o caracteres usando un "Códec". Hay dos principales:

1. **SL (Semantic Language)**: Legible por humanos (parecido a LISP). Es el estándar de facto para compatibilidad .
2. **LEAP**: Binario y comprimido. No es legible por humanos. Se usa en entornos de bajo rendimiento o móviles .

9.5.2. El Flujo de Trabajo (Pipeline)

El `ContentManager` es el componente del agente que gestiona esta traducción:

1. **Registro**: Al iniciar, el agente debe registrar qué Ontología y qué Lenguaje va a usar:

```
getContentManager().registerLanguage(new SLCodec());  
getContentManager().registerOntology(MiOntologia.getInstance());
```

2. **Enviar (Fill)**: El agente crea sus objetos Java (Beans), y el ContentManager los "inyecta" en el mensaje, traduciéndolos al lenguaje elegido (SL/LEAP).

- Método: `fillContent(ACLMessage msg, Element content)`.

3. **Recibir (Extract)**: El receptor toma el mensaje y el ContentManager reconstruye los objetos Java a partir de los bytes recibidos.

- Método: `extractContent(ACLMessage msg)`.

9.6. Herramientas de Desarrollo: Protégé

Escribir ontologías a mano es tedioso y propenso a errores.

- **Protégé:** Software de la Universidad de Stanford para dibujar ontologías gráficamente (clases, relaciones).
- **OntologyBeanGenerator (OBG):** Un plugin para Protégé que **genera el código Java automáticamente**. Crea tanto la clase `Ontology` (esquemas) como los Beans con sus getters y setters, listos para usar en JADE .
 - *Nota:* Requiere versiones antiguas de Protégé (3.4).

9.7 Intento de Entenderlo todo

Tienda de Discos, imagina esta situación:

- **Agente A (Comprador):** Quiere enviar un mensaje que diga: *"Quiero comprar el disco 'Thriller' de Michael Jackson"*.
- **Agente B (Vendedor):** Tiene que recibir ese objeto Java y entenderlo.

Paso 1: Definir las "Cosas" (Los Beans)

Primero creamos las clases Java que representan los datos. Estas son las piezas de LEGO que vamos a intercambiar.

1. El Concepto (El Sustantivo): Un `Vinilo` es un objeto pasivo. Implementa `Concept`.

```
import jade.content.Concept;

public class Vinilo implements Concept {
    private String titulo;
    private String artista;

    // Es OBLIGATORIO tener un constructor vacío y getters/setters
    public Vinilo() {}

    public String getTitulo() { return titulo; }
    public void setTitulo(String titulo) { this.titulo = titulo; }

    public String getArtista() { return artista; }
    public void setArtista(String artista) { this.artista = artista; }
}
```

2. La Acción (El Verbo): `Pedir` es algo que un agente hace. Implementa `AgentAction`. Fíjate que contiene un `Vinilo` dentro.

```
import jade.content.AgentAction;

public class Pedir implements AgentAction {
    private Vinilo viniloDeseado; // La acción lleva asociado el objeto

    public Pedir() {}

    public Vinilo getViniloDeseado() { return viniloDeseado; }
    public void setViniloDeseado(Vinilo vinilo) { this.viniloDeseado = vinilo; }
}
```

Paso 2: El Mapa (La Clase Ontology)

Aquí es donde le damos nombre a las cosas para JADE. Esta clase une tus clases Java (`Vinilo.class`) con un nombre semántico ("Vinilo").

```
import jade.content.onto.*;
import jade.content.schema.*;

public class TiendaOntology extends Ontology {
    // Patrón Singleton: Solo una instancia para todos
    public static final String ONTOLOGY_NAME = "Ontologia-Tienda";
    private static Ontology instancia = new TiendaOntology();

    public static Ontology getInstance() { return instancia; }

    private TiendaOntology() {
        super(ONTOLOGY_NAME, BasicOntology.getInstance()); // Hereda tipos básicos (String, Integer)

        try {
            // 1. Definimos el Concepto "Vinilo"
            add(new ConceptSchema("Vinilo"), Vinilo.class);

            // 2. Definimos la estructura interna (Slots) del Vinilo
            ConceptSchema cs = (ConceptSchema) getSchema("Vinilo");
            cs.add("titulo", (PrimitiveSchema) getSchema(BasicOntology.STRING));
            cs.add("artista", (PrimitiveSchema) getSchema(BasicOntology.STRING));

            // 3. Definimos la Acción "Pedir"
            add(new AgentActionSchema("Pedir"), Pedir.class);
        }
    }
}
```

```

        // 4. Estructura de Pedir (contiene un concepto Vinilo)
        AgentActionSchema as = (AgentActionSchema) getSchema("Pedir");
        as.add("viniloDeseado", (ConceptSchema) getSchema("Vinilo"));

    } catch (OntologyException oe) {
        oe.printStackTrace();
    }
}
}
}

```

Paso 3: El Agente Emisor (Cliente)

Ahora vamos a usarlo. El agente crea el objeto, lo rellena y el `ContentManager` lo convierte en mensaje.

```

// DENTRO DEL AGENTE COMPRADOR (setup o behaviour)

// 1. Registrar lenguaje y ontología (OBLIGATORIO)
ContentManager cm = getContentManager();
cm.registerLanguage(new SLCodec());
cm.registerOntology(TiendaOntology.getInstance());

// 2. Crear los datos (Java normal)
Vinilo v = new Vinilo();
v.setTitulo("Thriller");
v.setArtista("Michael Jackson");

Pedir accionPedir = new Pedir();
accionPedir.setViniloDeseado(v);

// 3. Preparar el mensaje ACL
ACLMessage msg = new ACLMessage(ACLMessage.REQUEST);
msg.addReceiver(new AID("Vendedor", AID.ISLOCALNAME));
msg.setLanguage(new SLCodec().getName()); // "FIPA-SL"
msg.setOntology(TiendaOntology.getInstance().getName()); // "Ontologia-Tienda"

// 4. LA MAGIA: Rellenar el contenido
// Las acciones siempre van envueltas en un "Action" genérico que indica QUIÉN lo
hace
Action wrapper = new Action(msg.getReceiver().next(), accionPedir);

try {
    cm.fillContent(msg, wrapper); // Traduce Java -> Lenguaje SL
}

```

```
    send(msg);  
} catch (Exception e) { e.printStackTrace(); }
```

Paso 4: El Agente Receptor (Vendedor)

El vendedor recibe un mensaje y extrae el objeto Java directamente.

```
// DENTRO DEL AGENTE VENDEDOR  
  
// 1. Registrar lo mismo que el emisor  
ContentManager cm = getContentManager();  
cm.registerLanguage(new SLCodec());  
cm.registerOntology(TiendaOntology.getInstance());  
  
ACLMMessage msg = receive();  
  
if (msg != null) {  
    try {  
        // 2. Extraer el contenido (Bytes -> Java)  
        ContentElement contenido = cm.extractContent(msg);  
  
        // 3. Verificar qué es  
        if (contenido instanceof Action) {  
            Action wrapper = (Action) contenido;  
  
            // Miramos qué acción hay dentro del sobre  
            if (wrapper.getAction() instanceof Pedir) {  
                Pedir peticion = (Pedir) wrapper.getAction();  
  
                // ¡YA TENEMOS EL OBJETO! Acceso directo a los datos  
                System.out.println("Me han pedido el disco: " +  
                    peticion.getViniloDeseado().getTitulo());  
            }  
        }  
    } catch (Exception e) { e.printStackTrace(); }  
}
```

Resumen Visual

1. **Beans:** Creas `class Vinilo` (Java puro).
2. **Ontology:** Creas `class TiendaOntology` para enseñar a JADE que `class Vinilo` equivale al concepto "Vinilo".
3. **ContentManager:** Es el traductor.

- **Emisor:** `fillContent()` (Java → Mensaje).
- **Receptor:** `extractContent()` (Mensaje → Java).

No envías textos sueltos, envías estructuras definidas. Si intentaras enviar un `Coche` en el campo de `Vinilo`, JADE daría error antes de salir.

10. Jade y Jess

10.1 Introducción y Arquitectura Conceptual

El objetivo es comprender cómo dotar a un agente de software de capacidad de **razonamiento declarativo**. Mientras que JADE proporciona la infraestructura para la comunicación y el ciclo de vida del agente, Jess actúa como el "cerebro" lógico.

10.1.1 El Modelo de Agente Relacional

Podemos visualizar el agente integrado mediante un modelo cibernético clásico:

- **Entorno (environment):** el mundo externo donde vive el agente
- **Sensores (Sensors):** mecanismos que reciben **perceptos** (entradas, como mensajes ACL)
- **Agente (Agent):** La entidad computacional. Aquí es donde ocurre la integración:
 - **JADE:** Gestiona la interacción con el entorno (envío/recepción de mensajes)
 - **Jess:** Implementa el módulo de decisión de forma declarativa (reglas lógicas).
- **Actuadores (Actuators):** mecanismos que ejecutan **acciones** sobre el entorno (ej. enviar una respuesta)

10.1.2 ¿Por qué integrar Jess?

Jess es un motor de reglas ligero y rápido escrito en Java. Permite construir software que "razona" usando conocimiento suministrado en forma de reglas declarativas, en lugar de código imperativo (if-then-else anidados).

- **Programación normal (Java/JADE sin Jess):** Es como dar órdenes a un soldado. Tienes que decirle paso a paso: "*Si ves un enemigo, levanta el arma. Si está a menos de 50 metros, dispara. Si no, espera*". Si la situación cambia y no prevista en tus instrucciones, el soldado se bloquea.
- **Motor de Inferencia (Jess):** Es el estratega. Tú no le dices qué hacer paso a paso. Tú le das **Reglas** ("Los enemigos cercanos son peligrosos") y **Hechos** ("Hay un tanque a 20 metros"). El estratega *deduce* por sí mismo: "Debo atacar".

NOTA

Es como usar un sistema experto de los de IA pero con agentes

10.2 Desafíos Técnicos de la Integración

Integrar un motor de inferencia en un agente JADE no es trivial debido a la naturaleza de la concurrencia en ambos sistemas.

10.2.1 El Modelo de Hilos

- **JADE:** Un agente JADE es, por principio, **monohilo (single-threaded)**. Esto simplifica la gestión del estado pero impone restricciones de bloqueo.
- **Jess (Clase Rete):** Para embeber Jess, instanciamos la clase `jess.Rete`. El método estándar de ejecución es `Rete.run()`.
- **El Problema:** El método `Rete.run()` ejecuta reglas consecutivamente y **bloquea el hilo de llamada** hasta que no quedan más reglas por disparar.

Advertencia Crítica: Si ejecutamos `Rete.run()` sin control dentro de un agente JADE, bloquearemos al agente completo. Si el razonamiento tarda mucho, el agente dejará de percibir mensajes y atender otras tareas.

10.2.2 Solución: Ejecución Intercalada

Para evitar el bloqueo total, utilizamos una variante del método `run` que acepta un límite de ciclos: `jess.run(int maxCycles)`. Esto permite que el agente razone "un poco", atienda otras tareas (como leer mensajes), y vuelva a razonar

10.3 Implementación: El `JessBehaviour`

La arquitectura recomendada encapsula el motor Jess dentro de un comportamiento cíclico de JADE (`CyclicBehaviour`).

10.3.1 Estructura del Comportamiento

El `JessBehaviour` debe gestionar la carga del motor y su ciclo de ejecución.

1. Inicialización (Constructor):

- Se crea la instancia de `jess.Rete`.
- Se carga el archivo de reglas (`.clp`) usando un `FileReader` y el parser `jess.Jesp`.

2. Ciclo de Ejecución (Método `action()`):

- Se define una constante `MAX_JESS_PASSES` (ej. 1) para limitar el tiempo de CPU dedicado a razonar en cada vuelta.
- Se invoca `jess.run(MAX_JESS_PASSES)`.
- **Gestión de inactividad:** Se compara el número de pases ejecutados con el máximo permitido. Si `ejecutados < maximo`, significa que el motor se detuvo porque **no hay más reglas aplicables**. En este caso, debemos bloquear el comportamiento (`block()`) para no consumir CPU inútilmente.

```
import jade.core.Agent;
import jade.core.behaviours.CyclicBehaviour;
import jade.lang.acl.ACLMessage;
import jess.*; // Importamos la librería de Jess

public class AgenteJess extends Agent {

    // El motor de inferencia (El cerebro)
    private Rete motor;

    protected void setup() {
        motor = new Rete();
        try {
            // Cargar las reglas desde un fichero .clp
            motor.batch("mis_reglas.clp");
            motor.reset(); // Llevar los hechos al estado inicial
        } catch (JessException e) { e.printStackTrace(); }

        // Añadimos el "Puente" entre JADE y JESS
        addBehaviour(new PuenteJadeJess());
    }

    private class PuenteJadeJess extends CyclicBehaviour {
        public void action() {
            try {
                boolean actividad = false;

                // 1. RECEPCIÓN: Traducir Mensajes JADE -> Hechos JESS
                ACLMessage msg = myAgent.receive();
                if (msg != null) {
                    // Convertimos el mensaje en un hecho para que Jess lo vea
                    // Ejemplo: (mensaje (contenido "fuego") (emisor "sensor1"))
                    String hecho = "(mensaje (contenido " + msg.getContent() + "))";
                    motor.assertString(hecho);
                    actividad = true;
                }
            }
        }
    }
}
```

```

    }

    // 2. RAZONAMIENTO: Ejecutar JESS paso a paso
    // ¡¡OJO AL EXAMEN!! Usamos run(1), NO run().
    // Esto dispara MÁXIMO 1 regla. Si hay 100 pendientes, las hará en
los siguientes ciclos.
    int reglasDisparadas = motor.run(1);

    if (reglasDisparadas > 0) {
        actividad = true;
    }

    // 3. COLABORACIÓN
    // Solo nos bloqueamos si NO hay mensajes Y Jess NO tiene reglas por
disparar.

    if (!actividad) {
        block();
    }

    } catch (JessException e) { e.printStackTrace(); }
    }
}

```

10.3.2 Inserción de Hechos (Facts)

Para que el motor razone, necesita datos. Debemos proveer métodos para insertar hechos y despertar al comportamiento si estaba dormido.

Ejemplo de método auxiliar `addFact` :

```

boolean addFact(String jessFact) {
    try {
        jess.assertString(jessFact); // Insertar hecho en Jess [cite: 164]
    } catch (JessException je) { return false; }

    if (!isRunnable()) restart(); // Si el comportamiento estaba bloqueado,
despertarlo [cite: 174]
    return true;
}

```

Nota: Es fundamental llamar a `restart()` (o un mecanismo equivalente para desbloquear) después de insertar un hecho, ya que nuevas premisas podrían activar nuevas reglas.

10.4 Intercambio de Información: De JADE a Jess

El flujo de información desde el entorno hacia el motor de reglas suele ocurrir a través de mensajes ACL.

10.4.1 Estrategia Básica: Traducción Manual

Se puede implementar un comportamiento (ej. `MsgListening`) que reciba mensajes ACL, extraiga su contenido y construya una cadena de texto (`String`) con formato de hecho Jess para inyectarlo mediante `addFact`.

10.4.2 Estrategia Avanzada: Shadow Facts (Java Beans)

Jess permite mapear objetos Java directamente a "Shadow Facts" en su memoria de trabajo, evitando la traducción manual a `String`.

- Se define un `deftemplate` en Jess apuntando a la clase Java: `(deftemplate ACLMessage (declare (from-class ACLMessage)))`.
- Se añade el objeto con `Rete.add(objeto)`.

El Problema de `ACLMessage`: La clase `ACLMessage` de JADE no cumple estrictamente con el estándar JavaBeans necesario para Jess. Específicamente, el campo `content` es un `StringBuffer`, pero sus métodos `get/set` trabajan con `String`. Esto hace que Jess falle al intentar inspeccionar el objeto automáticamente.

Solución: Crear una clase envoltorio o extendida, por ejemplo `MyACLMessage`, que corrija los accesos a los campos para ser compatible con la introspección de Java Beans.

10.5 Intercambio de Información: De Jess a JADE (Acciones)

Una vez que Jess ha razonado, el agente debe actuar (generalmente enviando un mensaje). Existen tres niveles de integración:

10.5.1 Post-procesamiento (Nivel Bajo)

El agente verifica la memoria de trabajo de Jess después de cada ejecución para ver si hay "hechos de salida" y actúa en consecuencia. Esto es pasivo y poco eficiente.

10.5.2 `Userfunction` (Nivel Medio - Recomendado)

Se crea una clase Java que implementa la interfaz `jess.Userfunction` (ej. `JessSend`). Esta clase actúa como un puente.

1. **Implementación:** El método `call` de la clase Java recibe argumentos desde Jess, construye un `ACLMessage` y usa `myAgent.send()` para enviarlo.
2. **Registro:** Se registra la función en el motor: `jess.addUserfunction(new JessSend(myAgent))`.
3. **Uso:** En el código Jess (reglas), se llama directamente: `(send ...)`.

10.5.3 Acceso Directo al Agente (Nivel Alto)

Se puede insertar el propio objeto `Agent` dentro del motor Jess usando `Rete.add(Agent)`. Esto permite invocar cualquier método público del agente (incluyendo `send`) directamente desde el código de las reglas usando la sintaxis de reflexión de Jess.

10.6 Conclusión de la Asignatura

Una vez más se demuestra que el grado presenta una dejadez considerable por parte de las "vacas sagradas" de la facultad. Y es que no puede ser: estas personas son reconocidísimas en sus ámbitos, publican 300 artículos, acuden a congresos internacionales... pero cuando llega el momento de pensar en sus alumnos y proporcionarles una bibliografía de calidad con la cual puedan aprender los conceptos de forma intuitiva, te sueltan un PDF de 2001 robado de otra universidad o presentaciones sin orden lógico alguno, llenas de palabras sueltas y esquemas incomprensibles.

Eso sí, si te quejas, la alternativa que te ofrecen es el libro de turno de 1990 escrito en alemán por un médico francés. Y apáñate tú para conseguirlo y entenderlo.

La situación es insostenible. Es frecuente encontrarse con:

- Documentación completamente desactualizada
- Materiales de dudosa procedencia sin adaptar al contexto actual
- Presentaciones caóticas sin estructura pedagógica
- Bibliografía obsoleta, inaccesible o en idiomas que nadie domina

¿Tan difícil es hacer las cosas bien? Otras carreras universitarias lo consiguen. No se pide la perfección, pero sí un mínimo de coherencia y actualización en los materiales docentes.

Considero que el grado debería aprender de otras titulaciones y mejorar urgentemente este aspecto. Con la jubilación progresiva de las viejas vacas sagradas de la universidad, espero sinceramente que esta situación mejore con el tiempo y que la nueva generación de profesorado traiga una renovación real en la calidad docente, no solo en la investigadora.

Resumen COMDIS

Tema 1

Un **paradigma** es un **patrón o modelo**. Una **aplicación distribuida** necesita que **dos o más procesos independientes intercambien información** y necesita **sincronización**, **hay que coordinar cuándo se envían y reciben**. **Paradigmas:**

- **Paso de Mensajes:** se basa en operaciones de entrada/salida similares a escribir en un archivo mediante el uso de primitivas básicas (`send` y `receive`).
- **Cliente Servidor:** es el modelo más conocido y **asimétrico**, donde hay una clara división de jerarquía. Los nodos que intervienen tienen funciones, objetivos y responsabilidades diferentes. El **cliente** adopta un **rol activo** donde inicia la comunicación y realiza peticiones al servidor, mientras que el **servidor** tiene un **rol pasivo** esperando pasivamente a recibir peticiones de los clientes para poder procesarlas. El objetivo es proporcionar una arquitectura que pueda ofrecer servicios facilitando la sincronización de eventos y permitiendo que los clientes sean ligeros centralizando recursos en el servidor.
- **P2P:** es una arquitectura **simétrica, homogénea y descentralizada**. No hay una división jerárquica como en cliente-servidor, los nodos llevan a cabo las mismas funciones y tienen las mismas responsabilidades. **Colaboran para conseguir objetivos comunes**. Además al no tener todos los recursos centralizados en un único servidor no existe un único punto de fallo y si falla un nodo los demás pueden seguir colaborando.
- **RPC:** abstracción que permite llamar a una función que está en otro ordenador. Emplea **stubs** generados con `rpcgen`
- **RMI:** evolución de RPC **orientada a objetos**. En RPC llamas a funciones y en RMI invoca a métodos de un objeto que vive en otra máquina.
- **ORB:** traductor universal independiente de plataforma y lenguaje, el estándar más famoso es **CORBA**.
- **Sistemas de Mensajes:** se caracterizan por el desacomplamiento temporal y espacial, permite que dos procesos se comuniquen sin tener que estar ambos de forma activa en la comunicación (**permite la comunicación asíncrona**).
- **Modelo punto a punto:** se deja un mensaje en la cola y el receptor lo recoge
- **Modelo publica suscribe:** se basa en eventos

Tema 2

El **api de sockets** es la interfaz de programación estándar para la comunicación entre procesos (IPC), es el nivel de **abstracción** más bajo que nos ofrece el sistema operativo.

TCP: En el modelo **Stream**, la comunicación es exclusiva entre dos procesos conectados.

```
//parte del servidor
SocketServer ss = new SocketServer(puerto);
Socket s = ss.accept();
DataInputStream in = new DataInputStream(s.getDataInputStream());
DataOutputStream out = new DataOutputStream(s.getDataOutputStream());
out.writeByte(5);
System.out.println(in.getBytes());

//parte del cliente
Socket s = new Socket ("ipservidor", puerto);
DataInputStream in = new DataInputStream(s.getDataInputStream());
DataOutputStream out = new DataOutputStream(s.getDataOutputStream());
out.writeByte(5);
System.out.println(in.getBytes());
```

UDP: En el modelo **datagrama** no hay conexión persistente. Cada paquete es independiente y lleva consigo la dirección de destino. Son como cartas independientes.

```
//parte del emisor
DatagramSocket ds = new DatagramSocket();
InetAddress receptor = InetAddress.getByName("ip");
byte[] data = "Hola".getBytes();
DatagramPacket dp = new DatagramPacket(data, data.length, receptor, puerto);
//permite enviar
ds.send(dp);

//parte del receptor
DatagramSockets ds = new DatagramSocket(puerto);
byte[] buffer = new byte[1024];
DatagramPacket dp = new DatagramPacket(buff, buff.length); //permite recibir
ds.receive(dp);
```

Por defecto **receive y read** son **bloqueantes**, por lo que impiden que hilo de ejecución avance y puede provocar que el programa se quede bloqueado de forma indefinida. Por ello usamos el método `socket.setSoTimeout(int milisegundos)` donde si se ha llamado a una operación bloqueante y no se ha recibido un mensaje, tras transcurrir el periodo de tiempo especificado en el método, **se lanza una excepción** que permite retomar el control.

Broadcast implica difusión a todos los dispositivos de una red mientras que **multicast** implica la difusión a todos los dispositivos de una subred.

Existen 2 formas de crear un hilo:

- **Heredando de Thread:**

```
public class MiHilo extends Thread(){
    @Override
    public void run(){
        ...
    }
}

Thread hilo = new MiHilo();
hilo.start();
```

- **Implementando Runnable:**

```
public class MiHilo implements Runnable(){
    @Override
    public void run(){
        ...
    }
}

MiHilo h = new MiHilo();
Thread hilo = new Thread(h);
hilo.start();
```

Para que un código sea **thread-safe** es importante proteger regiones críticas empleando **cerrojos**:

- **Bloque sincronizado:** protege un fragmento de código usando un objeto como cerrojo

```
synchronized(objetoCerrojo) {
    // Sección crítica
}
```

- **Método sincronizado:** esto usa como propio cerrojo al objeto, por lo que lo bloquea y no se puede usar ningún otro método mientras se usa el método sincronizado

```
public synchronized void metodo() { ... }
```

Además tenemos los siguientes métodos para poder manejar a los hilos:

- `wait()`: El hilo suelta el cerrojo y se duerme hasta ser avisado. Si se interrumpe, lanza `InterruptedException`.

- `notify()`: Despierta a un hilo aleatorio que esté esperando en ese objeto.
- `notifyAll()`: Despierta a **todos** los hilos esperando (más seguro para evitar bloqueos indefinidos).
- `start()`: Inicia la ejecución del hilo.
- `sleep(long ms)`: Pausa el hilo actual el tiempo indicado.
- `join()`: Hace que el hilo actual espere hasta que el hilo al que se llama termine (ej. el `main` espera a los trabajadores).
- `currentThread()`: Devuelve la referencia al hilo que está ejecutando esa línea de código.

Tema 3

Java RMI es la **evolución Orientada a Objetos de RPC**. **Java RMI es por naturaleza multihilo**. Cada solicitud de un cliente se procesa a través de un hilo independiente en el servidor, es importante que la **implementación del objeto remoto sea Thread Safe**. Para construir una app RMI disponemos de una metodología estricta basada en interfaces.

```
public interface InterfazRemota extends Remote{
    public String métodoRemoto() throws RemoteException;
}
```

Todos los métodos remotos deben lanzar `RemoteException` porque la red puede caerse, el servidor no existir o el stub perderse. Java obliga al programador a manejar estos fallos explícitamente.

```
public class ObjetoServidor implements InterfazRemota extends UnicastRemoteObject{

    public ObjetoServidor() throws RemoteException{
        super();
    }

    @Override
    public String métodoRemoto() throws RemoteException{
        System.out.println("H0la");
    }
}
```

El **objeto servidor** es aquel que implementa las operaciones definidas por la **interfaz remota**.

```
public class ServidorObjetos{
    public static void main(String[] args){
        try{
            LocateRegistry.createRegistry(puerto);
            ObjetoServidor os = new ObjetoServidor();
        }
    }
}
```

```

        Naming.rebind("rmi://ip:puerto/Servicio", os);
    }catch (Exception e){
        e.printStackTrace();
    }
}
}

```

El **servidor de objetos** es el encargado de iniciar el registro RMI, instanciar al **objeto servidor** y publicarlo en el registro.

En el lado de cliente simplemente hacemos un cast a la interfaz remota de la referencia que devuelve el método `Naming.lookup(url)`.

```

public class Cliente{
    public static void main(String[] args){
        try{
            InterfazRemota ir = (InterfazRemota)
Naming.lookup("rmi://ip:puerto/Servicio");
            ir.métodoRemoto();
        }catch(Exception e){
            e.printStackTrace();
        }
    }
}

```

A la hora de desarrollar una aplicación RMI simple necesitamos la siguiente colocación de archivos:

- **Servidor:** `InterfazRemota.class`, `ObjetoServidor.class`, `ServidorObjetos.class`, `Skell.class`
- **Cliente:** `InterfazRemota.class`, `Cliente.class`, `Stub.class`.

El `stub.class` y `skell.class` son simplemente los proxies de cliente y de servidor generados tras ejecutar `rmic ObjetoServidor.class`. Cuando un cliente quiere ejecutar un método, lo ejecuta sobre el stub, el **stub realiza el marshalling** (serialización) convirtiendo los argumentos del método en un flujo de bytes que envía a través de la red. El **skeleton realiza el unmarshalling** (deserialización), reconstruye los argumentos originales a partir de los bytes recibidos y llama al método en el **objeto servidor real**. Para retornar los valores del método se realiza el proceso contrario.

Tema 4

El **stub downloading** soluciona el problema de tener que redistribuir de forma manual el `Stub.class` cuando se realizaba un cambio en la implementación del **objeto remoto**. Para ello almacenamos el

stub en un **servidor http**, y cuando el cliente ve que no tiene el stub lo descarga automáticamente del servidor http. Para esto a la hora de arrancar el **servidor de objetos** tenemos que indicar varias propiedades:

- **Codebase:** indica la url del servidor http.
- **Hostname:** indica la ip pública del servidor
- **Policy:** es el archivo de permisos de seguridad

```
java -Djava.rmi.server.codebase=http://www.miservidor.com/clases/ -  
Djava.rmi.server.hostname=mi.ip.publica -Djava.security.policy=java.policy
```

El **gestor de seguridad** es imprescindible porque al usar **stub downloading** estás ejecutando código descargado de una red externa, lo que supone un gran riesgo. Por ello usamos

`System.setSecurityManager(new RMISecurityManager());` encargado de gestionar la seguridad consultando el archivo `java.policy`. En este archivo definimos los **permisos que va a tener la aplicación** (conectarse al registro rmi en el puerto 1099, descargar stubs mediante el puerto 80 y conexiones efímeras).

En java RMI existe la posibilidad de realizar una **comunicación bidireccional (RMI Callbacks)** para solucionar el problema de que el cliente sondee de forma constante al servidor saturando la red (**polling**). Para implementar esto necesitamos que el **objeto servidor** tenga un **array** donde almacene las referencias remotas de los clientes y un método para poder registrarlos.

```
public class ObjetoServidor implements InterfazRemotaServidor extends  
UnicastRemoteObject{  
    private vector<InterfazRemotaCliente> clientes;  
  
    public ObjetoServidor throws RemoteException{  
        super();  
    }  
  
    @Override  
    public addClienteCallback(INTERfazRemotaCliente c){  
        clientes.add(c)  
    }  
  
    private llamarCliente(){  
        clientes.get(0).serllamado();  
    }  
}
```

La **colocación de archivos con RMI Callback** es:

- **Servidor:** `IntServer.class`, `ImplServer.class`, `Server.class`, `IntClient.class`,
`StubClient.class`, `SkellServer.class`.

- **Cliente:** `IntClient.class`, `ImplCliente.class`, `Client.class`, `IntServer.class`, `StubServer.class`, `SkellClient.class`.

Por último, para poder transmitir objetos a través de la red mediante el uso de rmi estos deben de ser serializables, serializar es un mecanismo que permite convertir un objeto en una secuencia de bytes para que se pueda enviar por la red. Para enviar un objeto como argumento en RMI **debe implementar la interfaz** `java.io.Serializable`, si no lo hace, RMI lanzará una excepción al intentar enviarlo. Además **todos los atributos del objeto deben de ser serializables** (primitivos, strings o implementar serializable). Si hay datos que no quieres enviar debes marcarlos con la palabra clave `transient`.

Tema 5

- **Napster:** tenía una arquitectura híbrida dónde consultabas a un servidor central quién tenía el archivo deseado y después obtenías la ip del usuario deseado y te conectabas al cliente para descargarlo. El problema era que **el servidor era un único punto de fallo, un cuello de botella**.
- **Gnutella:** P2P puro, dónde las consultas **se realizaban por inundación** y el mensaje de petición tenía un **TTL** máximo para saltar entre nodos. **El problema era que había un tráfico de red y no garantizaba encontrar el archivo**.
- **Kaaza/Fastrack:** introducen el concepto de **superpeers**, nodos con un nivel jerárquico superior al resto por su potencia y buena conexión, que actuaban como pequeños servidores napster. Los nodos normales subían sus listas de archivos al Super-peer y le preguntaban a él
- **Freenet:** en los anteriores sistemas distribuidos la identidad no era anónima, pero aquí si que lo era mediante un mecanismo de **descubrimiento incremental** dónde la **repuesta viaja siguiendo el camino inverso a la consulta**. Es imposible saber si un nodo está iniciando o consumiendo una petición o simplemente **reenviando la de otro**.
- El **problema del gorrón consiste en que la mayoría de usuarios no subían nada y solo descargaban**.

El problema de **gnutella** y otros predecesores P2P era que no garantizaban encontrar el archivo aunque exista o era ineficientes. Por ello surgen las **DHT (Tablas Hash Distribuidas)**, que son un tipo de sistema distribuido descentralizado que ofrece un servicio de búsqueda similar a una tabla hash dónde **los datos están distribuidos entre los nodos de la red, cada nodo es responsable de un pequeño rango de claves** y cualquier nodo puede encontrar de forma eficiente $O(\log(N))$ a otro nodo que tiene el dato que busca, sin necesidad de un servidor central. Tenemos los siguientes algoritmos de DHT:

- **Chord:** los nodos de la red forman un anillo lógico donde cada archivo se almacena en el nodo cuyo **identificador sea igual o inmediatamente mayor al identificador del archivo**. Además cada nodo tiene una **tabla de dedos** en la que contiene las direcciones de los nodos ubicados a distancias potencia de 2. El problema es que es un **anillo lógico y no físico por lo que se puede producir mucha latencia**.
- **Pastry:** intenta solucionar el problema de chord, enrutando los nodos mediante claves parciales, dónde si soy el nodo `abcde` y busco al `ghjk`, primero busco al nodo que empieza por `g` y si hay varios candidatos escojo al más cercano geográficamente.

- **Can:** ignora el anillo y códigos y trata a la red como un mapa de coordenadas gigante, donde si estás en el (0,0) y quieres enviar un archivo al (1,1), le pasa el mensaje a tu vecino de la derecha arriba.

Tema 6

El problema de las arquitecturas RPC, CORBA, RMI y DCOM es que se intentaban conectar a puertos aleatorios poco comunes que eran **bloqueados por los firewalls** y además se requería que ambas máquinas a comunicar usasen la misma tecnología o lenguaje.

Después esto dio lugar al concepto de **Tunneling** a través de **Java Servlets**. La técnica consistía en encapsular las peticiones de datos dentro de tráfico HTTP estándar, permitiendo que atravesase firewalls. Los servlets actuaban como pequeños programas en el servidor que **escuchaban peticiones en el puerto 80**. Sin embargo el html no está estructurado y requiera que el cliente al otro lado parsee de forma manual la información estructurado.

SOA es un paradigma para construir sistemas distribuidos basado en la integración de servicios autónomos, débilmente acoplados e interoperables. Permite que aplicaciones desarrolladas en tecnologías heterogéneas se comuniquen mediante estándares abiertos, combinando el transporte universal (**HTTP**) con un formato de datos estructurado (**XML**).

Está formada por 3 elementos:

- **Proveedor:** crea y expone el servicio
- **Registro:** directorio donde se registran los servicios para ser encontrados
- **Consumidor:** buscar y usa el servicio

Funcionamiento:

- El proveedor crea el servicio y su contrato (**WSDL**) accesible vía URL
- Registra el servicio en el directorio UDDI para que sea visible
- El consumidos busca en el registro y obtiene la url del wsdl
- Con el wsdl el consumidor genera automaticamente stubs y proxies
- Por utlimo se lanza la petición encapsulada en SOAP a través de http

WSDL es un archivo XML jerárquico dividido en dos bloques:

- **Descripción abstracta:** se define la interfaz funcional sin importar el lenguaje de proramación, define los tipos de datos, los tipos de mensajes, el portType que funciona como un agrupador estilo interfaz de java y las operaciones, que definen la accion conectando un mensaje de entrada con uno de salida.

Jerarquía: types → componen message → usados en operation → agrupados en portType .

- **Descripción concreta:** define la conexión física. Indica el `binding`, que es el tipo de protocolo asociado al `portType`, normalmente SOAP sobre HTTP. `service` que agrupa a los puertos y `port` que una un `binding` a un puerto.

SOAP es un estándar XML para invocar servicios, no tiene estado y es **unidireccional** y **síncrono**. Todo el mensaje SOAP es un XML con 3 partes:

- **Envelope:** raíz obligatoria que encapsula todo
- **Header:** es opcional contiene metadatos, puede ser procesado por nodos **intermediarios**.
- **Body:** es la carga útil con los datos de negocio. Solo para el **destinatario final**.

Los atributos de la cabecera permiten controlar el flujo a través de intermediarios:

- `role`: define qué nodo debe leer la cabecera (`next`, `ultimateReceiver`)
- `mustUnderstand`: si es `true` y el nodo no entienda la etiqueta, **debe fallar**. Además los mensajes SOAP pueden tener dos estructuras:
- **Document:** le envías datos
- **RPC:** pides que se ejecute una función, simula una llamada a código.

El viaje del mensaje desde tu código Java hasta el servidor remoto:

1. **Código Cliente:** Llama al método local (`stub.sumar(5,5)`).
2. **Stub Cliente:** Delega en el **Motor SOAP** (JAX-WS).
3. **Motor SOAP:** Serializa a XML (SOAP) y lo pasa al **Cliente HTTP**.
4. **Red:** El mensaje viaja por HTTP (POST).
5. **Router SOAP (Servidor):** Recibe el XML, lo parsea y decide a qué servicio llamar.
6. **Skeleton/Stub Servidor:** Deserializa el XML a objetos Java.
7. **Implementación:** Se ejecuta el código real (`public int sumar...`).

```
//interfaz
package com.ejemplo;

@WebService
public interface Calculadora{
    @WebMethod
    int sumar(int a, int b);
}
```

```
//implementación
package com.ejemplo;

@WebService(endpointInterface="com.ejemplo.Calculadora") //esto sirve para que solo
se publiquen los métodos definidos en la interfaz y no los de la implementación.
public class CalculadoraImpl implements Calculadora{
```

```

        public int sumar(int a, int b){
            return a+b;
        }
    }
}

```

```

// publicador
public class Publicador{
    public static void main(String[] args){
        Endpoint.publish("http://localhost_8080/miCalculadora", new
        CalculadoraImpl());
    }
}

/*Endpoint.publish
- Arranca: Inicia un servidor HTTP ligero interno en el puerto 8080.
- Genera: Crea el WSDL dinámicamente en memoria.
- Expone: Habilita la escucha de peticiones SOAP en esa URL.
*/

```

```

//cliente
public class Cliente {
    public static void main(String[] args) {
        try {
            // 1. URL del WSDL (El Manual)
            URL wsdlURL = new URL("http://localhost:8080/miCalculadora?wsdl");

            // 2. QName (Identificador Único: Namespace + ServiceName)
            QName qname = new QName("http://ejemplo.com/", "CalculadoraImplService");

            // 3. Crear la Fábrica (Service)
            Service service = Service.create(wsdlURL, qname);

            // 4. Obtener el Proxy (Stub)
            Calculadora proxy = service.getPort(Calculadora.class);

            // 5. Invocación Remota (Marshalling -> Red -> Unmarshalling)
            System.out.println("Resultado: " + proxy.sumar(10, 20));

        } catch (Exception e) { e.printStackTrace(); }
    }
}

```

Método	Función Técnica
<code>URL(...?wsdl)</code>	Apunta a la ubicación del contrato. Sin el <code>?wsdl</code> al final, no funciona.
<code>QName</code>	Nombre Cualificado. Identifica al servicio exacto dentro del XML del WSDL. Se forma con el <code>TargetNamespace</code> (paquete invertido) y el <code>ServiceName</code> .
<code>Service.create()</code>	La Fábrica. Descarga el WSDL, lo valida y prepara la conexión.
<code>getPort()</code>	El Proxy (Stub). Crea un objeto local falso. Convierte tus llamadas Java (<code>.sumar</code>) en mensajes XML SOAP (Marshalling) y los envía por HTTP.

Tema 7

Un **sistema de mensajes** o **MOM** es un **middleware** que permite el desacoplamiento temporal y estructural mediante una conexión única al MOM. **RabbitMQ** es un middleware de mensajería que emplea **AMQP**, que es un protocolo asíncrono que garantiza el envío fiable de mensajes en colas seguras hasta que el receptor se conecte o cumpla los criterios para recibirlo.

Cuando el emisor envía un mensaje a un consumidor este siempre pasa por un **exchange** que decide a qué cola mandar el mensaje en base a unos criterios:

- **Default:** la routing key del mensaje es el nombre de la cola destino, si coincide el exchange envía el mensaje a esa cola.
- **Fanout:** el exchange ignora la routing key del mensaje y envía el mensaje a todas las colas conectadas al exchange.
- **Direct:** el exchange compara la routing key con la binding key de la cola para saber si puede enviarla a esa cola.
- **Topic:** similar a direct pero permite coincidencias parciales entre colas mediante el uso de wildcards. Un mensaje con clave `a.b.c` podría entrar a una cola que escucha `a.*` o `a.b.#`
- **Header:** ignora la routing key e inspecciona los **metadatos** para compararlos con la binding key.

```
//productor
ConnectionFactory cf = new ConnectionFactory();
cf.setHost("localhost");
try{
    Connection cn = cf.newConnection();
    Channel ch = cn.createChannel();
    //queueDeclare(nombreCola, durable, exclusive, autoDelete, arguments);
    ch.queueDeclare("COLA", false, false, false, null);

    String msg = "H0la";
    //basicPublish(exchange, routingKey, metadatos,body);
    ch.basicPublish("", "COLA", null,msg.getBytes());
}
```

```
//consumidor
ConnectionFactory cf = new ConnectionFactory();
cf.setHost("localhost");
try{
    Connection cn = cf.newConnection();
    Channel ch = cn.createChannel();
    ch.queueDeclare("COLA", false, false, false, null);

    DeliverCallback deliverCallback = (consumerTag, delivery) -> {
        String message = new String(delivery.getBody(), "UTF-8");
        System.out.println("Recibido: " + message);
    };

    // basicConsume(queue, autoAck, deliverCallback, cancelCallback)
    channel.basicConsume("COLA", true, deliverCallback, consumerTag -> { });

}
}
```

Tipos de exchange:

- **Fanout:**

```
// 1. Declaramos el exchange de tipo FANOUT
channel.exchangeDeclare("exchange_deportes", "fanout");

// 2. Publicamos.
String mensaje = "GOL DEL EQUIPO LOCAL";
channel.basicPublish("exchange_deportes", "", null, mensaje.getBytes());
```

```
channel.exchangeDeclare("exchange_deportes", "fanout");

// Creamos una cola aleatoria, exclusiva y temporal (se borra al desconectar)
String nombreCola = channel.queueDeclare().getQueue();

// BINDING: "Conecta mi cola temporal al exchange de deportes"
channel.queueBind(nombreCola, "exchange_deportes", "");

// Consumimos
channel.basicConsume(nombreCola, true, deliverCallback, consumerTag -> {});
```

- **Direct:**

```
// 1. Tipo DIRECT
channel.exchangeDeclare("exchange_logs", "direct");

// 2. Enviamos un error grave
String mensaje = "Fallo crítico en base de datos";
String severidad = "error"; // Esta es la Routing Key
channel.basicPublish("exchange_logs", severidad, null, mensaje.getBytes());
```

```
channel.exchangeDeclare("exchange_logs", "direct");
String nombreCola = channel.queueDeclare().getQueue();

// BINDING: "Solo envíame mensajes si la routing key es EXACTAMENTE 'error'"
channel.queueBind(nombreCola, "exchange_logs", "error");

channel.basicConsume(nombreCola, true, deliverCallback, consumerTag -> {});
```

- **Topic:**

```
channel.exchangeDeclare("exchange_noticias", "topic");

String mensaje = "El Madrid gana la liga";
// Clave: Categoría.Subcategoría.Equipo
channel.basicPublish("exchange_noticias", "deportes.futbol.madrid", null,
mensaje.getBytes());
```

```
// BINDING: El asterisco (*) sustituye a UNA palabra (el equipo)
// Recibirá: deportes.futbol.madrid, deportes.futbol.barca
channel.queueBind(nombreCola, "exchange_noticias", "deportes.futbol.*");
```

```
// BINDING: La almohadilla (#) sustituye a CUALQUIER cantidad de palabras
// Recibirá: deportes.futbol.madrid, deportes.tenis, deportes.waterpolo.femenino
channel.queueBind(nombreCola, "exchange_noticias", "deportes.#");
```

- **Header:**

```
channel.exchangeDeclare("exchange_archivos", "headers");

Map<String, Object> cabeceras = new HashMap<>();
cabeceras.put("formato", "pdf");
cabeceras.put("tipo", "informe");
```

```
AMQP.BasicProperties props = new AMQP.BasicProperties.Builder()
    .headers(cabeceras)
    .build();

channel.basicPublish("exchange_archivos", "", props, mensaje.getBytes());
```

```
Map<String, Object> bindingArgs = new HashMap<>();
bindingArgs.put("x-match", "all"); // Deben coincidir TODAS las reglas
bindingArgs.put("formato", "pdf");

channel.queueBind(nombreCola, "exchange_archivos", "", bindingArgs);
```

Tema 8

Un agente es un sistema informático situado en un entorno, capaz de realizar acciones de forma **autónoma** y flexible para cumplir los objetivos de su dueño. Un agente es:

- **Autónomo:** opera sin intervención directa, tiene control sobre su estado interno y sus acciones.
- **Reactividad:** percibe su entorno y responde a cambios rápidamente
- **Proactividad:** no solo reacciona, toma la iniciativa para cumplir objetivos a largo plazo
- **Habilidad Social:** interactúa con otros agentes o humanos (coopera, negocia o compite).
- **Movilidad:** el agente tiene la capacidad de viajar de nodo en nodo de la red preservando su estado.

La principal diferencia entre los sistemas multiagente y las redes P2P es la **homogeneidad**, el nivel de autonomía e inteligencia. En una red P2P los nodos que intervienen son todos homogéneos, su comportamiento es **rígido y reactivo**, todos los nodos realizan las mismas funciones y **colaboran entre sí**. Por el otro lado, los agentes son **heterogéneos**, cada uno se especializa en realizar una tarea o función y puede **competir o colaborar** con otros agentes. Además su comportamiento es **proactivo y autónomo**, no solo reactivo.

Los agentes se comunican siguiendo los estándares **FIPA-ACL**. Un mensaje FIPA tiene una estructura tipo carta que incluye:

- **Performative:** El tipo de acto (request, inform, agree)
- **Sender/Receiver:** quién envía y quién recibe
- **Content:** el contenido real de la comunicación
- **Language & Ontology:** que idioma y diccionario se usa para entender el contenido
- **Protocol:** a qué conversación pertenece esto

Si recibes un mensaje que no entiendes, **debes** responder con `not-understood`, si usas un acto comunicativo debes cumplir su definición oficial y no estás obligado a implementar todos los

protocolos, solo el subconjunto que necesites.

Jade es un **framework** de código abierto que facilita la creación de sistemas distribuidos multiagente, está escrito en Java y cumple con la FIPA. Tenemos los siguientes agentes disponibles para desarrollar el sistema:

- **RMA:** es la consola principal, controla la plataforma, mata y crea a otros agentes.
- **DF:** son las páginas amarillas del sistema, permite que los agentes se encuentren por lo que hacen y no por su nombre.
- **Sniffer Agent:** muestra un diagrama visual de quién envía mensajes a quién
- **Introspector Agent:** permite ver el **estado interno** de un agente, qué comportamientos está ejecutando, la cola de mensajes pendientes y el valor de sus variables
- **Log Manager Agent:** gestiona los logs de la plataforma en tiempo de ejecución. Permite cambiar qué nivel de detalle se guarda sin tener que reiniciar el programa.
- **Dummy Agent:** un agente manual para enviar mensajes de prueba y comprobar si tu sistema responde correctamente.

El ciclo de vida de un agente es el siguiente:

- **Nacimiento (`setup`):** se ejecuta una sola vez al arrancar. Aquí inicializas variables y, lo más importante, **añades los primeros comportamientos** (`addBehaviour`) a la "piscina" de tareas.
- **El Bucle Principal (El corazón del Agente):** el agente entra en un bucle `while` que dura toda su vida. ¿El agente ha muerto (`doDelete` llamado)? Si ha muerto sale del bucle y va a `takeDown()`, si no continúa.
- **Ejecución de Comportamientos:** el agente coge el siguiente comportamiento activo de su lista, ejecuta el método `b.action()` de ese comportamiento (solo se ejecuta un paso, no todo el proceso si es largo).
- **Verificación:** Pregunta `b.done()` . Si devuelve **True** el comportamiento ha terminado y se elimina de la lista, si devuelve **False** el comportamiento sigue vivo y se queda en la lista para la siguiente vuelta
- **Muerte (`takeDown()`):** se ejecuta antes de desaparecer. Aquí cierra conexiones a bases de datos, te desregistras de las páginas amarillas, etc.

```
import jade.core.Agent;

public class MiAgente extends Agent {

    // 1. SETUP: Equivale al "main". Se ejecuta al nacer.
    protected void setup() {
        System.out.println("Hola, soy el agente " + getAID().getName());
        // Aquí se añaden los comportamientos (el cerebro)
        addBehaviour(new MiComportamiento());
    }
}
```

```
// 2. TAKEDOWN: Se ejecuta justo antes de morir (limpieza).
protected void takeDown() {
    System.out.println("Adios mundo cruel.");
}
}
```

Un agente **JADE funciona en un SOLO HILO de Java**. ¿Cómo puede hacer varias cosas a la vez (escuchar mensajes y calcular datos)? Usando **comportamientos colaborativos, no apropiativos**. Para realizar esto, el programador debe definir cuando el comportamiento debe **ceder CPU** de forma voluntaria a otros comportamientos (normalmente cuando se usan operaciones bloqueantes, esto se explica más abajo).

1. `myBehaviour.block()` (Enfoque Colaborativo)

- **Mecanismo:** Es un bloqueo **lógico**. Marca el comportamiento como "en espera" y devuelve el control al planificador del agente.
- **El Hilo (Thread): No se detiene.** El agente aprovecha para ejecutar otros comportamientos concurrentes.
- **Despertador:** El comportamiento se reactiva automáticamente cuando llega un **nuevo mensaje** a la cola del agente.
- **Uso:** Es la forma correcta de esperar mensajes dentro del método `action()`.

2. `myAgent.blockingReceive()` (Enfoque Bloqueante)

- **Mecanismo:** Es una llamada **síncrona**.
- **El Hilo (Thread): Se bloquea físicamente.** Detiene por completo el flujo de ejecución del hilo Java del agente.
- **Consecuencia:** El agente se "congela" totalmente. Ningún otro comportamiento puede ejecutarse hasta que llegue el mensaje esperado.
- **Uso:** Evitar en comportamientos cíclicos. Útil solo en la inicialización (`setup()`) o fases lineales estrictas.

Tipo	Clase Java	Descripción	<code>done()</code> devuelve...	Ejemplo
OneShot	<code>OneShotBehaviour</code>	Se ejecuta una sola vez y termina.	<code>true</code>	Enviar un email de bienvenida.
Cíclico	<code>CyclicBehaviour</code>	Nunca termina. Se repite infinitamente.	<code>false</code>	Escuchar mensajes entrantes.
Waker	<code>WakerBehaviour</code>	Se ejecuta una vez tras un tiempo de espera (Temporizador).	<code>true</code>	"Despiértame en 10 segundos".
Ticker	<code>TickerBehaviour</code>	Se ejecuta repetidamente cada X tiempo	<code>false</code>	Comprobar el precio de una acción cada

Tipo	Clase Java	Descripción	done() devuelve...	Ejemplo
		(Metrónomo).		minuto.
Complejo	FSMBehaviour, etc.	Máquinas de estados. Cambia de tarea según lo que pase.	Depende	Lógica compleja de negocio.

```
//enviar un mensaje acl
ACLMessage msg = new ACLMessage(ACLMessage.INFORM); // 1. Performative
msg.addReceiver(new AID("AgenteB", AID.ISLOCALNAME)); // 2. Receptor
msg.setLanguage("Castellano");
msg.setContent("Está lloviendo"); // 3. Contenido
send(msg); // 4. Enviar
```

```
//recibir un mensaje acl
ACLMessage msg = receive(); // Mira si hay mensaje
if (msg != null) {
    // Procesar mensaje
} else {
    block(); // ¡IMPORTANTE!
}
```

Tema 9

Una ontología **es un contrato semántico**. Es una especificación explícita de una conceptualización. Define el vocabulario (tipos, propiedades y relaciones) de un dominio específico sin ambigüedad.

El **ContentManager** es el componente del agente encargado de la **traducción semántica** entre los objetos Java (lógica interna) y el contenido textual de los mensajes (transmisión por red).

Funciona apoyándose en dos elementos que debes registrar: un **Lenguaje** (la sintaxis, ej. FIPA-SL) y una **Ontología** (el vocabulario). Su utilidad se resume en dos métodos:

1. **fillContent()** : Convierte tus objetos Java a texto para enviar un mensaje.
2. **extractContent()** : Convierte el texto del mensaje recibido de nuevo a objetos Java.

Escribir ontologías a mano es tedioso y propenso a errores.

- **Protégé**: Software de la Universidad de Stanford para dibujar ontologías gráficamente (clases, relaciones).
- **OntologyBeanGenerator (OBG)**: Un plugin para Protégé que **genera el código Java automáticamente**. Crea tanto la clase `Ontology` (esquemas) como los Beans con sus getters y

setters, listos para usar en JADE .

Tema 10

Jess es un **motor de reglas** ligero y rápido escrito en Java. Permite construir software que razona usando conocimiento suministrado en forma de reglas declarativas, en lugar de código imperativo.

Un agente JADE es, por principio, **monohilo (single-threaded)**. Esto simplifica la gestión del estado pero impone restricciones de bloqueo. Para embeber Jess, instanciamos la clase `jess.Rete`. El método estándar de ejecución es `Rete.run()`. El método `Rete.run()` ejecuta reglas consecutivamente y **bloquea el hilo de llamada** hasta que no quedan más reglas por disparar. Para evitar el bloqueo total, utilizamos una variante del método `run` que acepta un límite de ciclos: `jess.run(int maxCycles)`. Esto permite que el agente razone "un poco", atienda otras tareas (como leer mensajes), y vuelva a razonar